Robot-Assisted Bed Baths for Patients with Limited Mobility

by

Jonathan Sanjay

Supervisor: Dr. Andrew Goldenberg
April 7th, 2025

B.A.Sc. Thesis

Division of Engineering Science
UNIVERSITY OF TORONTO

B.A.Sc. Thesis

Robot-Assisted Bed Baths for Patients with Limited Mobility

University of Toronto
Division of Engineering Science
Jonathan Sanjay
Supervised by Dr. Andrew Goldenberg
April 7, 2025

Abstract

As global healthcare systems face rising demands due to aging populations and increasing rates of mobility impairment, there is a critical need for advanced robotic solutions to support Activities of Daily Living (ADLs). Among these, bathing is one of the most physically demanding tasks for caregivers and one of the most dignity-sensitive tasks for patients. Existing robot-assisted bed-bathing systems remain largely semi-autonomous, requiring human oversight and intervention. This thesis contributes to closing this gap by developing and validating a minimum viable product (MVP) for an autonomous bed-bathing system, implemented and tested in a simulation environment.

Using ROS 2, Gazebo, and MoveIt, the system integrates perception, inverse kinematics, and motion planning modules within a modular control architecture. A custom-designed collaborative robotic arm (cobot) equipped with a simplified gripper and depth camera autonomously identifies a point on the patient's body surface using 3D point cloud data, transforms it into a valid end-effector pose, and plans a safe, collision-free motion to execute a simulated wiping action. The control pipeline is orchestrated through a launch-based ROS workflow, enabling seamless coordination of system components in real time.

Simulation-based experiments demonstrate that the MVP reliably completes the wiping task with sub-centimeter accuracy, maintains contact forces within clinically safe limits, and exhibits robust performance under varying conditions. Although limited to single-point cleaning in a simplified environment, the system lays a strong foundation for future expansion.

This work advances the state of assistive robotics by delivering a modular, extensible framework for autonomous hygiene assistance. It demonstrates the feasibility of integrating perception, decision-making, and control in a robotic caregiver context. Future work includes expanding to full-surface coverage, incorporating multimodal perception, and addressing the ethical and clinical dimensions of real-world deployment. The long-term vision is to empower caregivers and enhance patient dignity through human-centered robotic autonomy.

Acknowledgements

I would like to express my deepest gratitude to my grandmother, whose unwavering strength, sacrifice, and love shaped the person I am today. She raised me with unwavering devotion and taught me by example that hard work, humility, and quiet perseverance can carry you through anything. Everything I have achieved, I owe to her.

To my parents, thank you for pouring your hearts into my growth and believing in me unconditionally. Your constant support, even through the most difficult seasons, has shaped both my character and my calling. I carry your hopes with me in everything I do, and I strive always to make you proud.

I am also sincerely grateful to Professor Andrew Goldenberg for supporting me throughout this research and for the opportunity to pursue this project under his supervision. His guidance and encouragement have been instrumental to this work.

Above all, I thank God for His grace, guidance, and faithfulness throughout this journey. Time and again, He has led me through the Red Sea, opening a way forward when there seemed to be none. Every opportunity, strength, and breakthrough I have experienced is a testament to His presence in my life. This work, like all that I do, is for His greater glory.

Table of Contents

List of Figures

1 Introduction

The growing demographic of aging individuals and the increasing prevalence of mobility impairments necessitate the development of advanced solutions to support Activities of Daily Living (ADLs), with bathing being of paramount importance. In North America, millions of people face significant challenges in performing essential hygiene tasks due to physical disabilities. Bathing, as a critical ADL, is intrinsically linked to both physical health and emotional well-being. However, traditional approaches to providing bed baths for individuals with limited mobility remain labor-intensive, often undermining patient dignity and comfort while exacerbating caregiver burden and burnout.

Current robotic systems, such as RABBIT [1] and comparable assistive technologies, predominantly operate as semi-autonomous tools that heavily depend on human intervention during the bathing process. This dependency underscores a critical gap in the development of fully autonomous robotic systems capable of independently executing bed baths. Addressing this gap is crucial for enhancing patient care, safeguarding the dignity of individuals, and alleviating the physical and emotional strain on caregivers in an increasingly overburdened healthcare system.

The focus of this research is to design and validate a fully autonomous, simulation-based robotic system for delivering bed baths to individuals with limited mobility. The central objectives of this study are:

1. Robotic Modeling: Develop a virtual environment using the Robot Operating System (ROS) and Gazebo to simulate the complete bed bathing process, enabling fully autonomous robotic operations.

2. Autonomous Decision-Making: Design algorithms for autonomous decision-making that dynamically assess and respond to patient needs, such as skin integrity and comfort, during the bathing process.

3. Performance Validation: Conduct simulation-based evaluations to assess the system's efficacy, focusing on metrics such as task completion time, operational safety, and patient satisfaction.

This research aims to bridge the technological divide between current semi-autonomous systems and fully autonomous robotic solutions. By employing state-of-the-art robotics, sensor integration, and machine learning methodologies, this project seeks to advance a system that not only addresses functional requirements but also prioritizes the quality of life for both patients and caregivers.

2 Literature Review

Traditional caregiving methods for bed bathing are labor-intensive, place significant physical and emotional burdens on caregivers, and can often lead to discomfort and indignity for patients. Theoretical frameworks for developing assistive technologies in this domain emphasize multimodal perception, compliant control mechanisms, and human-centered design. A comprehensive understanding of these principles is essential for bridging the gap between robotic capabilities and the nuanced requirements of caregiving tasks.

2.1 RABBIT: A Robot-Assisted Bed Bathing System with Multimodal Perception and Integrated Compliance

Current robotic systems, such as RABBIT and its predecessors, demonstrate significant advancements in automating caregiving processes [1]. The RABBIT system leverages RGB and thermal imaging to distinguish between dry, soapy, and wet skin regions accurately, ensuring precise washing, rinsing, and drying actions. Previous systems introduced wearable robots and inflatable end-effectors yet failed to incorporate essential caregiving practices like the application of soap or drying techniques [1]. Such limitations underscore the necessity for advanced perception and compliant control mechanisms to replicate human caregiving practices effectively

2.1.1 Advances in Multimodal Perception

Distinguishing between wet, dry, and soapy skin presents unique challenges due to the translucency of water and variability in skin tones and textures. RGB-Thermal imaging has proven to be a robust solution in overcoming these challenges. The RABBIT system introduces a domain-specific dataset, the Synthetic Bathing Perception (SBP), which trains segmentation models to identify these features under diverse conditions [1]. This dataset, combined with transformer-based architectures like CMX-SRA, achieves high segmentation accuracy, setting a benchmark for future robotic perception models.

2.1.2 Compliant Control Mechanisms in Bed Bathing

A critical component of robot-assisted bed bathing is ensuring safe and comfortable physical human-robot interaction (pHRI). RABBIT employs dual compliance mechanisms—software-based force modulation and hardware compliance through a custom-designed end-effector ("Scrubby") [1]. This dual approach enables the system to adapt to varied body contours and exert appropriate pressure during bathing tasks. Comparatively, earlier systems focused

on position tracking without incorporating force modulation, which is essential for patient safety and comfort.

2.1.3 Design and Evaluation of the RABBIT System

The RABBIT system integrates human caregiving-inspired motion primitives, ensuring that its actions closely mimic those of trained caregivers. The system performs systematic washing, rinsing, and drying motions validated through a user study involving individuals with and without mobility limitations [1]. High ratings for comfort and task performance in the study highlight the system's potential to replace or supplement human caregivers effectively. However, feedback from participants also points to areas for improvement, such as addressing inaccessible body regions and integrating real-time adaptive planning [1].

2.1.4 Research Gaps and Future Directions

Despite its advancements, the field of robot-assisted bed bathing continues to face challenges. Current systems, including RABBIT, require enhancements to accommodate full-body bathing and adapt to involuntary patient movements. Moreover, incorporating real-time feedback mechanisms to dynamically adjust force and motion trajectories will be critical in improving user experience and task efficiency. Addressing these gaps will facilitate the development of more sophisticated, autonomous caregiving robots.

2.2 I-Support: A Robotic Platform of An Assistive Bathing Robot for the Elderly Population

2.2.1 Technological Innovations in the I-Support System

Motorized Chair for Accessibility: A commercial motorized chair was adapted with three degrees of freedom (DOF), providing lateral, vertical, and rotational movement capabilities [2]. This design ensures safe and efficient user transfer into and out of the bathing area, accommodating the spatial constraints of typical bathrooms. The adjustable height and rotational axis further enhance the accessibility and usability of the system.

Human–Robot Interaction Module: The HRI component employs Kinect V2 RGB-D cameras to enable gesture and audio-based communication [2]. By integrating audio–gestural recognition, users can interact naturally with the robot using predefined commands for tasks like "wash legs" or "scrub back." The multimodal communication system ensures real-time responsiveness, enhancing user engagement and safety during operation.

Soft Robotic Arm: A bio-inspired manipulator combines flexible fluidic actuators and tendon-driven mechanisms to provide compliance and adaptability [2]. This arm can elongate, bend, and adjust its stiffness dynamically, making it capable of safely interacting with the user's body. The manipulator ensures gentle scrubbing and washing while adhering to safety guidelines and minimizing the risk of injury.

2.2.2 Multimodal Perception and Context Awareness

Environmental Awareness: Sensors such as Amphiro devices and CubeSensors provide data on water flow, temperature, air humidity, and illumination [2]. These inputs are crucial for ensuring operational safety and adapting the system's actions to user comfort.

User-Specific Adaptation: A smartwatch-based system tracks user preferences, including scrubbing patterns, sensitive areas to avoid, and medical conditions. This information is stored in a central database, enabling personalized bathing procedures that cater to individual needs.

2.2.3 Motion Planning and Real-Time Adaptation

The robotic system's motion planning relies on visual and depth data from Kinect sensors. By leveraging point-cloud data and body part segmentation, the system dynamically adjusts the robot's end-effector position during tasks like washing and scrubbing [2]. Real-time adaptability ensures precision in handling curved and deformable surfaces, such as the user's back or legs. Additionally, imitation learning methods, such as dynamic motion primitives, incorporate caregiver expertise to produce motions that mimic human care.

2.2.4 Clinical Validation Studies

Validation Round I: Conducted in dry conditions, this phase focused on evaluating human–robot communication using audio and gestural commands. Participants, aged 67 to 81, exhibited high performance in executing audio commands (up to 98% accuracy), with slightly lower success rates for gestural commands due to their complexity [2]. System command recognition rates (CRR) reached 83.5% during leg-washing tasks, demonstrating the reliability of multimodal interaction.

Validation Round II: This phase assessed the fully functional system under wet conditions, including tasks like scrubbing and rinsing. Audio–gestural command recognition reached 86%, while user satisfaction scores averaged 63.8 out of 100 on the System Usability Scale (SUS), indicating "good" usability [2]. Participants appreciated the intuitive interface and

system responsiveness, highlighting its potential for enhancing independence.

2.2.5 Research Gaps and Future Directions

While the I-Support system demonstrates significant advancements in assistive robotics, several areas require further exploration. These include enhancing gesture recognition algorithms for greater robustness, improving adaptability for users with cognitive impairments, and addressing complex bathing scenarios, such as cleaning hard-to-reach body areas. Future research could also focus on optimizing cost-effectiveness to make the system accessible to a broader population.

2.3 Towards an Assistive Robot that Autonomously Performs Bed Baths for Patient Hygiene

2.3.1 Key Components of the Robotic System

Cody is equipped with two 7-degree-of-freedom (DOF) compliant robotic arms with series elastic actuators (SEAs), ensuring safe physical interaction with human skin [3]. A tilting Hokuyo laser range finder and Point Grey Firefly camera capture point-cloud data and images of the target area. This combination facilitates operator selection of cleaning areas via a visual interface. A spatula-shaped, 3D-printed end effector wrapped in a moistened towel mimics the "bath mitt" technique used by nurses, allowing for effective debris removal and comfort.

2.3.2 Autonomous Wiping Behavior

The robot employs equilibrium point control (EPC) to perform wiping motions [3]. EPC is a form of impedance control that enables the robot to maintain precise contact forces (1–3 N) during cleaning, minimizing risks of injury or discomfort. The wiping algorithm follows these steps:

1. The operator selects the cleaning area via an interface.
2. The robot adjusts its end effector to the selected area, initiates contact and begins wiping motions.
3. A Cartesian-space equilibrium point (CEP) guides the end effector along pre-defined trajectories while adjusting to body curvature [3].

2.3.3 Experimental Evaluation and Results

The robot removed over 96% of debris in all trials, with complete cleaning achieved on the forearm and thigh. Small amounts of residue remained on the upper arm (3.05%) and shank (2.08%), attributed to variations in cleaning area curvature [3]. Average force application across trials remained below 3 N, with low variability, ensuring safety and comfort during operation. Tests showed a root mean square (RMS) error of 0.62 cm (x-axis) and 0.72 cm (y-axis) between operator-selected points and the robot's actions, demonstrating high precision in targeting [3].

2.3.4 Research Gaps and Future Directions

While promising, the system has limitations and areas for further development. Current capabilities are limited to surfaces aligned with gravity. Future iterations should incorporate adaptive end effectors for cleaning areas with varied orientations. Additionally, a fully autonomous system would require advanced algorithms for body part recognition and segmentation.

2.4 System for Bedside Assistance Integrating a Robotic Bed and a Mobile Manipulator

The system described in this paper combines a robotic bed (Autobed) and a mobile manipulator (PR2) to assist patients with daily living tasks in bed. This approach addresses the challenges of traditional stationary assistive systems, leveraging collaboration between heterogeneous robotic components to improve physical and perceptual capabilities.

2.4.1 System Components and Design

Robotic Bed (Autobed): The Autobed is a custom robotic bed derived from a commercial hospital bed, equipped with sensors and actuators to perform a range of tasks [4]. The Autobed features three degrees of freedom, allowing adjustments to height, backrest angle, and leg support. These features enable optimal positioning for task execution. A pressure-sensitive mat detects the patient's pose and estimates the body's center of mass, facilitating accurate positioning. This method overcomes occlusion issues commonly encountered with visual-only systems. The Autobed integrates a simplified control system using ROS, providing users with precise adjustments via a web-based interface.

Mobile Manipulator (PR2): Two 7-degree-of-freedom arms allow precise manipulation, though their payload capacity is limited to lightweight tools [4]. A mobile base enables the PR2 to

navigate around the bed, enhancing reach and access to task-specific locations. A Kinect V2 camera provides visual input for locating the Autobed and monitoring task execution.

2.4.2 Task Planning and Collaboration

The robotic bed positions the user, and the PR2 adjusts its base to align with task-relevant areas. Tasks such as feeding, applying lotion, or adjusting blankets are optimized through pre-defined configurations. Once the robots are positioned, users can manually control fine movements via a web interface. This hybrid model reduces cognitive load while preserving user oversight.

2.4.3 Evaluation and Results

The collaborative system achieved a 100% success rate for tasks such as leg bathing and arm care, compared to 33% without robotic bed adjustments [4]. Adjustments to the Autobed improved PR2's reach and task success, particularly in scenarios requiring precise positioning. The system was tested in the home of Henry Evans, a user with severe quadriplegia. Henry successfully completed tasks like mouth wiping, forehead cleaning, and blanket adjustment across all trials [4]. He highlighted the system's ease of use and reduced mental effort during autonomous phases. However, tasks involving delicate maneuvers, such as yogurt wiping, were noted as challenging [4].

2.4.4 Research Gaps and Future Directions

Enhancing the precision of human pose estimation using advanced sensors or algorithms could further improve task success rates. The system currently focuses on a limited set of tasks. Future iterations could address more complex activities of daily living (ADLs). Developing cost-effective and compact versions of the system would increase its accessibility to broader populations.

2.5 VTTB: A Visuo-Tactile Learning Approach for Robot-Assisted Bed Bathing

The VTTB (Visuo-Tactile Transformer-based imitation learning for Bathing assistance) approach stands out by addressing a persistent challenge in robot-assisted care: accurately sensing and interacting with the human body in a contact-rich environment. By leveraging multimodal sensing and Transformer-based imitation learning, the VTTB system advances the field of assistive robotics.

2.5.1 Multimodal Sensing for Improved Perception

The VTTB methodology integrates visual and tactile sensing, creating a robust multimodal system for accurate surface interaction during bed bathing [5]. Visual sensing, powered by depth cameras, captures the 3D contours of body surfaces, while tactile sensing detects localized contact information, including geometry and force. This dual approach overcomes limitations associated with using either modality alone, such as visual occlusion or inadequate tactile precision, enabling the robot to track body contours effectively and perform safe cleaning actions

2.5.2 Transformer-Based Imitation Learning

The core innovation of VTTB lies in its Transformer-based imitation learning framework. This model utilizes cross-attention mechanisms to fuse visual and tactile data streams, allowing the robot to focus on task-relevant features dynamically [5]. Unlike conventional neural networks, Transformers excel at long-range dependencies, making them ideal for capturing intricate relationships between sensory modalities. VTTB's ability to generalize across multiple surface geometries highlights its versatility in addressing diverse care scenarios.

2.5.3 Experimental Validation and Results

The system was validated using a Baxter robot and medical manikins to simulate real-world bed bathing tasks. The robot achieved a high task success rate by maintaining continuous contact with body surfaces while following contours accurately [5]. Experiments confirmed the significant contributions of both visual and tactile inputs, with notable performance drops observed when either modality was excluded. The system demonstrated robustness across unseen body parts and real human subjects, achieving over 80% success rates in trials.

2.5.4 Safety and User-Centric Design

Safety is central to the VTTB approach, with the system maintaining contact forces within a safe boundary of 10 Newtons [5]. The robot's ability to adapt its actions based on tactile feedback ensures user comfort and minimizes the risk of harm during operation. This design aligns with international standards for assistive robotics and paves the way for broader adoption in care settings.

2.5.5 Research Gaps and Future Directions

While VTTB offers significant advancements, areas for improvement include expanding training datasets to include more diverse real-world scenarios and human subjects, enhancing the system's ability to handle complex bathing tasks requiring three-dimensional coverage and addressing limitations related to the physical constraints of the Baxter robot, such as its bulkiness and maneuverability. Future iterations could integrate advanced path-planning techniques and explore applications in other ADLs (Activities of Daily Living).

3 Methods

3.1 Overview of Methodology

This chapter outlines the comprehensive methodological approach employed in this research, aimed at developing a fully autonomous robotic system capable of performing bed baths for individuals with limited mobility. The primary objective of this section is to clearly articulate and justify the procedures, techniques, and decisions underpinning the simulation-based design and evaluation of the robotic system. This detailed presentation of methodology ensures replicability, facilitates validation, and establishes a clear logical connection to the subsequent results and discussions.

Given the complexity and ethical considerations involved in testing assistive robotic systems directly with human subjects, this research utilizes a sophisticated simulation-based approach leveraging the Robot Operating System (ROS) integrated with the Gazebo simulator. Simulation methodologies are increasingly prevalent within robotic research as they allow for comprehensive validation of robotic behaviors in controlled yet realistically modeled environments, significantly reducing risk and enhancing experimental reproducibility [6], [7].

This methods section is systematically structured to guide the reader through the logical progression of the research methodology:

- **Simulation Environment Setup:** Establishes the simulation environment by detailing the configuration of ROS and Gazebo, justification of software and hardware choices, and modeling components including virtual representations of the hospital bed, human patient, robotic arm, sensors, and gripper.
- **Robotic System Design:** Provides a rigorous technical description of the robotic hardware and end-effector, with clear justification for the chosen robotic components. Particular emphasis is placed on the compliance mechanisms and safety features integrated into the robot design, aligning with best practices in human-robot interaction (HRI) [8].
- **Autonomous Decision-Making Algorithms:** Describes the autonomous algorithms developed for real-time decision-making. This subsection includes the rationale behind selecting specific methods for sensor fusion and dynamic adaptability to varying patient conditions, guided by recent advancements in robotic caregiving literature [9], [10].
- **Experimental Procedure and Simulation Testing:** Outlines the structured protocols followed during simulation tests. It explicitly defines different operational sce-

narios tested and the criteria used for performance evaluation, ensuring clarity and transparency in the experimental design.

- **Data Collection and Processing:** Details the procedures for systematic data acquisition during simulations, including the types of data collected and the methods employed for subsequent data analysis and visualization, emphasizing the methodological rigor required for reproducible and reliable scientific inquiry.

By structuring the methodology around these clearly defined subsections, this chapter not only facilitates ease of understanding and replication by researchers but also aligns the methods explicitly with the research objectives stated earlier. Subsequent results and their interpretation, as discussed in later chapters, are therefore firmly grounded in the clearly documented and justified methodological choices outlined herein.

3.2 Simulation Environment Setup

3.2.1 Justification for Simulation-Based Approach

Given the complex and sensitive nature of robotic caregiving tasks, particularly those involving direct human-robot interaction, simulation-based approaches offer significant benefits over physical prototyping. Simulation allows rigorous and safe testing of robotic behaviors, iterative refinement of algorithms, and accurate modeling of dynamic interactions without the costs and risks associated with initial hardware deployment. ROS and Gazebo were selected specifically due to their extensive adoption in robotics research, strong community support, and proven capabilities in accurately simulating robotic dynamics and sensor interactions [2] [5]. Gazebo provides realistic physical interaction simulations and accurate sensor emulation, making it ideal for modeling human-robot caregiving scenarios [5].

3.2.2 Software and Hardware Components

The simulation relies primarily on three software tools:

- **Robot Operating System (ROS2 Humble):** ROS2 is used for its robust modular architecture, allowing efficient management of complex robotic operations through nodes, topics, and service-based communication, essential for coordinating the multiple subsystems involved in bed-bathing tasks.

- **Gazebo Simulator:** Gazebo provides an accurate physical environment simulation, crucial for evaluating robotic performance under realistic caregiving conditions, including physical interactions, sensor feedback, and dynamic environment interactions [5].

- **MoveIt! Motion Planning Framework:** MoveIt! enables efficient and collision-free motion planning and trajectory optimization, vital for ensuring safe robotic movements near sensitive areas of a patient's body.

3.2.3 Modeling the Virtual Environment

The virtual environment, defined in the `patient_env.world` file [A.1], represents a simplified yet functional testbed designed to validate the core interactions of the autonomous bed-bathing system.

- **Patient Surface:** Instead of a fully detailed hospital bed, the current environment includes a flat platform serving as the patient surface. This minimal setup allows for straightforward interaction and robot reachability while focusing on validating core system functionality rather than environmental fidelity.
- **Human Patient Model:** A single human model, `human_horizontal`, is used to represent a bedridden patient lying on their back. This simplification enables consistent evaluation of the robot's motion planning and interaction strategies in a typical bed bath posture.
- **Caregiving Object (Sponge):** The sponge used in the simulation is a simplified cube model. While it does not yet reflect the physical properties or shape of a real sponge, it serves as a placeholder to validate grasping, manipulation, and motion trajectories associated with simulated cleaning actions.
- **Robot and Workspace Layout:** The cobot is placed to the left of the patient on a fixed base attached to the ground. A secondary table next to the robot holds the sponge. This setup emulates a minimal caregiving station configuration and supports testing of the robot's reachability and interaction with caregiving tools.

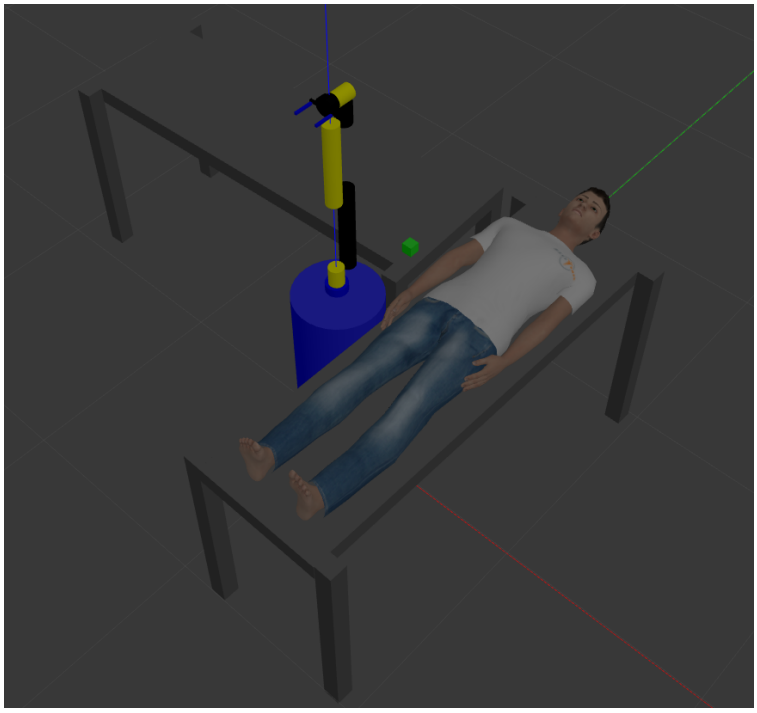

Figure 1: Simulated Gazebo environment featuring the robotic arm, hospital bed, and human model

This initial setup prioritizes functional validation over visual or physical realism. It reflects a minimum viable product approach to simulation, where simplified elements are intentionally used to iteratively test and refine the robot's core perception, planning, and control capabilities.

3.2.4 Setup of Sensors

Accurate sensor modeling is critical in simulating realistic caregiving interactions, and the following sensors were integrated within Gazebo:

- **Depth Camera (RGB-D Camera):** Modeled after the widely utilized Kinect sensor, the simulated depth camera (camera.xacro) [A.1] provides both RGB and depth data streams. Such sensors are pivotal in robotic caregiving systems, enabling essential perception tasks such as object recognition, patient body segmentation, and motion tracking. The camera's specific placement and orientation above the robotic workspace maximize visual coverage of the caregiving scene, thus reducing occlusion and ensuring robust perception performance under varying patient and environmental conditions.
- **Force/Torque Sensors:** Integrated within the robotic manipulator and gripper, these

virtual sensors measure interaction forces accurately. Force feedback is critical for ensuring compliance with safety regulations during physical human-robot interactions, minimizing the risk of injury or discomfort [3]. The inclusion of force sensors enables testing of sophisticated force modulation algorithms, ensuring the robot applies gentle yet effective force during bathing tasks, crucial for patient comfort and safety.

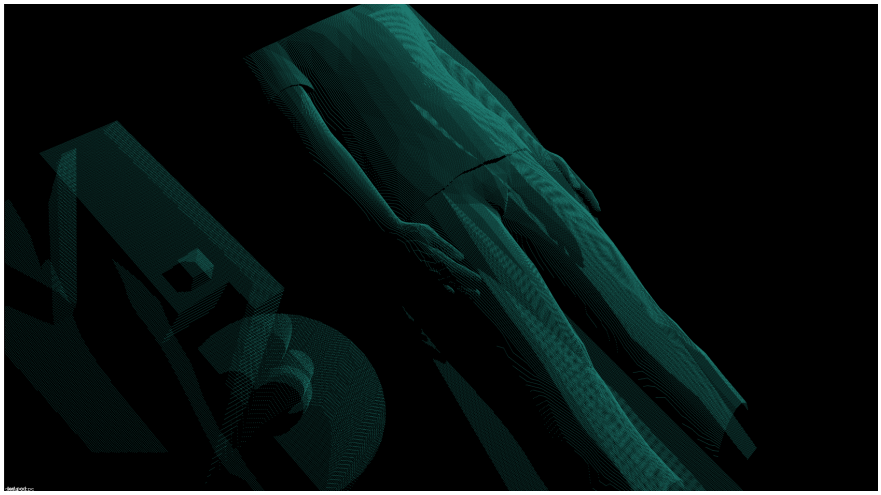

Figure 2: End-effector perspective: RGB-D camera view of the simulated scene

3.3 Robotic System Design

3.3.1 Robotic Arm Configuration and Mechanical Design

The robotic system developed for autonomous bed-bathing tasks employs a custom-designed collaborative robotic arm, or cobot, which has been specifically engineered to safely interact with human patients. The cobot's mechanical structure, kinematic configuration, and integrated compliance mechanisms were deliberately selected to balance precise motion control with gentle, compliant interactions, essential in caregiving contexts.

The robotic arm's detailed kinematic and dynamic characteristics are defined in the URDF (Unified Robot Description Format) and Xacro (XML Macros) files within the simulation package. Specifically, the files `custom_cobot_gazebo.xacro`, `custom_gripper.xacro`, and `cobot_gripper_camera.xacro` precisely specify the robotic arm, gripper mechanism, and integrated camera system used for perception [A.1].

The cobot features a 6-degree-of-freedom (DoF) articulated configuration, enabling extensive maneuverability and flexibility necessary for covering diverse patient body contours. Each joint is modeled with realistic joint limits and dynamic properties, including friction

and damping parameters, enhancing simulation fidelity. These specifications facilitate sophisticated motion trajectories and adaptive manipulation capabilities essential for effective caregiving tasks.

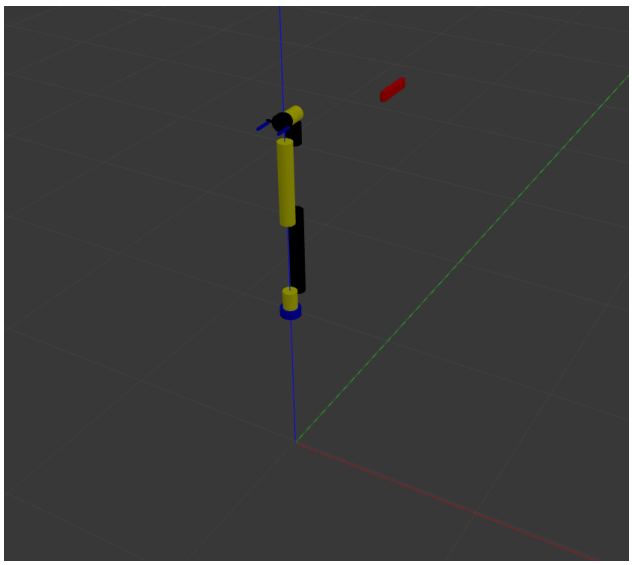

Figure 3: Robotic arm configuration within the Gazebo simulation environment

3.3.2 End-Effector (Gripper) Design

The custom gripper described in `custom_gripper.xacro` is specifically designed for caregiving operations, particularly the gentle grasping and manipulation of caregiving tools, such as sponges. It features a compliant parallel-jaw mechanism with integrated force-feedback sensors modeled virtually in Gazebo. The compliance in the gripper's design is critical, allowing the robotic system to modulate grasping force dynamically, thereby ensuring gentle and safe interactions with caregiving objects and indirectly with the patient's body surfaces.

Additionally, the gripper design incorporates a camera mounting interface, defined in `cobot_gripper_camera.xacro`. This camera integration directly on the end-effector enables precise visual perception in proximity to caregiving actions, critical for tasks involving fine positional adjustments and real-time feedback control.

3.3.3 Integrated Perception Capabilities

The gripper-integrated camera defined in `cobot_gripper_camera.xacro` enhances the robot's perceptual capabilities significantly. This RGB-D camera provides real-time visual and depth feedback, essential for tasks such as object detection, patient segmentation, and visual servoing. Its placement directly on the gripper significantly reduces occlusion issues common in

caregiving scenarios, enabling accurate positional feedback during interactions with complex, variable human anatomy.

Moreover, this integrated perception system allows advanced sensor-fusion techniques, combining tactile feedback from force sensors and visual/depth information, crucial for ensuring safe, compliant interactions that respect both patient comfort and operational safety criteria.

3.3.4 Motion Planning and Control

The robotic system leverages ROS's MoveIt! framework (defined in the `cobot_moveit_config` folder) [A.1], chosen explicitly for its robust and flexible motion planning capabilities. MoveIt! enables efficient collision checking, trajectory optimization, and motion execution strategies, crucial for safely navigating around the patient and caregiving environment.

The inverse kinematics (IK) solutions used for motion planning are detailed in the `cobot_ik` package, facilitating rapid computation of joint configurations corresponding to desired end-effector positions. Efficient IK solutions ensure real-time adaptability and responsiveness in caregiving tasks. Particularly, custom IK solutions implemented in the provided code optimize joint angles for comfort-oriented poses, minimizing patient discomfort and ensuring safe operational boundaries are strictly maintained.

3.3.5 Material and Structural Considerations

While simulated, the structural materials and mechanical properties used in the virtual models have been informed by standard collaborative robotic arm designs employed in healthcare applications, often utilizing lightweight but rigid structural alloys such as aluminum or carbon-fiber composites. These material choices are justified due to their high strength-to-weight ratios, reducing inertia and enabling safer, more responsive movements, critical in caregiving robotics [1].

3.3.6 Justification and Benchmarking Against Existing Systems

The custom cobot arm and gripper design choices were benchmarked against existing robotic systems described in the literature, such as RABBIT, I-Support, and Cody [1] [2] [3]. These systems commonly utilize compliant mechanisms and sensor integration strategies similar to those developed here, validating the appropriateness and effectiveness of the chosen robotic configurations. By incorporating proven design principles from existing robotic caregiving research, this robotic system ensures both innovation in autonomous operation and adherence to established safety and compliance standards critical for healthcare robotics.

3.4 Autonomous Decision-Making and Control Architecture in Simulation

This section details the autonomous decision-making algorithms and overall control architecture of the robot-assisted bed-bathing system, as implemented in the ROS-Gazebo simulation. We describe how the system launches and orchestrates various components, how the MoveIt motion planning framework integrates with the Gazebo simulator and ROS control, the inverse kinematics (IK) solution approach (`cobot_ik`), and the processing of 3D sensor data (point clouds) to select target surface points for wiping. We also explain how those target points are transformed into end-effector poses and how the controller managers, motion planners (OMPL), and ROS publishers/subscribers work together to execute the planned motions. The presentation is organized into logical parts, each focusing on a key aspect of the control architecture, with supporting mathematical formulations and technical references.

3.4.1 System Launch and Orchestration Logic

The simulation is started using a ROS 2 launch file (`spawn_moveit.launch.py`) that orchestrates the entire system startup. This launch file brings up the Gazebo simulation environment, inserts the robot model and sensors, and initializes the MoveIt motion planning components in one coordinated process. In sequence, the launch script will:

1. **Start the Gazebo simulator:** Gazebo is launched with a specified world (e.g., a hospital bed scene) and with the ROS 2 time synchronized (using simulation time).
2. **Spawn the robot model:** The robot's URDF/Xacro description (including the manipulator and any attached sensors like an RGB-D camera) is spawned into Gazebo. This involves loading the URDF parameters and using a `spawn_entity` utility to instantiate the robot in the simulated world at the desired pose.
3. **Initialize ROS 2 control and state publisher:** The `robot_state_publisher` node is started to publish the robot's joint TF frames, and the `ros2_control` controller manager is launched. The robot's URDF contains `ros2_control` plugin definitions (transmissions, joints, and hardware interfaces) so that when Gazebo spawns the robot, a controller manager is created within Gazebo to handle joint control. Essential controllers, such as a joint state broadcaster (publishing the `/joint_states` topic) and joint trajectory controllers for the arm, are loaded and activated.
4. **Launch MoveIt and planning components:** In parallel with the above, the launch

file starts the MoveIt `move_group` node and (optionally) an RViz visualization. The MoveIt configuration (from the `cobot_moveit_config` package) is loaded, including the robot's kinematic model (URDF) and semantic description (SRDF) which define joint limits, planning groups (e.g. the entire 6-DOF arm as one group), and any predefined pose configurations. These parameters are provided to `move_group` on startup. The `move_group` node (also known as the MoveIt planning server) then initializes the motion planning pipeline (using OMPL by default) and the planning scene monitor. If an RViz session is launched, it loads a MoveIt plugin for interactive visualization of the robot and environment. The launch file ensures that simulation time is used throughout (so MoveIt and controllers are in sync with Gazebo's clock).

All these steps are encapsulated so that running the one launch file will bring up the simulated robot and the planning framework together. In summary, `spawn_moveit.launch.py` automates the setup of Gazebo (with the robot and sensors spawned and physics running), the ROS 2 control interfaces (controller manager and joint controllers), and the MoveIt planning server. This orchestrated startup is critical for the autonomous operation: it provides the foundation where the robot's current state is continuously fed into MoveIt and where planning outputs can be sent to the robot in simulation. Once launched, the system is ready to perceive the environment and execute motion plans as described next.

3.4.2 MoveIt Integration with Gazebo and ROS Control

The MoveIt motion planning framework is tightly integrated with the Gazebo simulator and ROS control to enable motion planning and execution on the simulated robot. MoveIt's `move_group` node serves as the central planning and decision engine: it maintains the robot's model and state, computes motion plans, and interfaces with the robot's controllers for execution. Several key integration points ensure MoveIt and Gazebo work in unison:

1. **Robot Model and Planning Scene:** MoveIt loads the robot's URDF and SRDF on startup (from the configuration package), so it knows the kinematic structure (links, joints) and allowed motions of the robot. MoveIt also maintains a Planning Scene - a data structure representing the world and robot. A Planning Scene Monitor is started in `move_group`, which subscribes to the robot's joint states and other sensors to keep the scene updated [11]. Specifically, the planning scene monitor listens on the `/joint_states` topic (published by Gazebo's `joint_state_broadcaster`) to update the robot's current joint configuration in real time. It also uses TF transforms to be aware of the robot's pose in the world frame [11]. For example, the robot's base link may be fixed in the world, but if it were mobile, MoveIt would rely on TF (e.g.,

a map to `base_link` transform) to know the global position [11]. In our case, the base is typically fixed, but the sensor (camera) might have a TF relative to the robot. The `robot_state_publisher` (launched with the URDF) continuously provides all link frames via TF, which MoveIt uses for collision checking and for transforming goal points from sensor frame to robot frame.

2. **Motion Planning Pipeline (OMPL):** MoveIt's planning pipeline is configured to use the OMPL (Open Motion Planning Library) as the default planner for generating trajectories. The MoveIt configuration specifies one or more planning algorithms and when a motion plan is requested (for example, to move the end-effector to a target pose), `move_group` formulates a Motion Plan Request that includes the start state (current joint angles from the planning scene), the goal (desired end-effector pose or joint state), and any constraints. The OMPL interface then searches for a collision-free path in the robot's joint space that achieves the goal [11]. By default, MoveIt uses sampling-based planning via OMPL's algorithms, which randomly explore the 6-DOF configuration space and connect start to goal (subject to collision avoidance and joint limits) [11]. The result of planning is a trajectory – a sequence of time-parameterized waypoints for each joint that smoothly moves the arm to the target. MoveIt automatically time-parameterizes the path (ensuring joint velocity and acceleration limits are respected) so that the output is a feasible trajectory rather than just a geometric path [11].

3. **Controller Interface and Execution:** Once a trajectory is planned, MoveIt's `move_group` uses a FollowJointTrajectory action interface to send the trajectory to the robot for execution [11]. In ROS 2, the JointTrajectoryController (part of `ros2_control` and running in Gazebo) provides this action server. `move_group` acts as a client, sending the trajectory message and then waiting for result. The crucial aspect is that MoveIt itself does not execute joint commands directly; it hands off the trajectory to the controller manager in Gazebo [11]. The `ros2_control` framework then interpolates and applies control commands at each timestep to the simulated joints. As the trajectory is executed, the `joint_state_broadcaster` publishes the changing joint angles, which the planning scene monitor in MoveIt uses to track progress. If the execution completes successfully (the action server returns a success result), `move_group` knows the robot has reached the target. This feedback loop closes the chain: MoveIt plans using the state from Gazebo, and Gazebo's controller executes commands from MoveIt.

4. **Synchronization and Coordination:** Because all components use ROS 2 and simulation time, the planning and execution are synchronized. For instance, MoveIt ensures

that the trajectory it sends is feasible in the current state (if the robot moved due to physics or was nudged, the updated `joint_states` are taken into account just before execution). Additionally, any changes in the environment in Gazebo (e.g., a moved object or a patient model) could be reflected in the planning scene if sensors report them, enabling dynamic re-planning if needed. In the current implementation, the environment is mostly static (bed and patient model fixed), but the infrastructure allows reacting to changes if sensors/scene updates are incorporated.

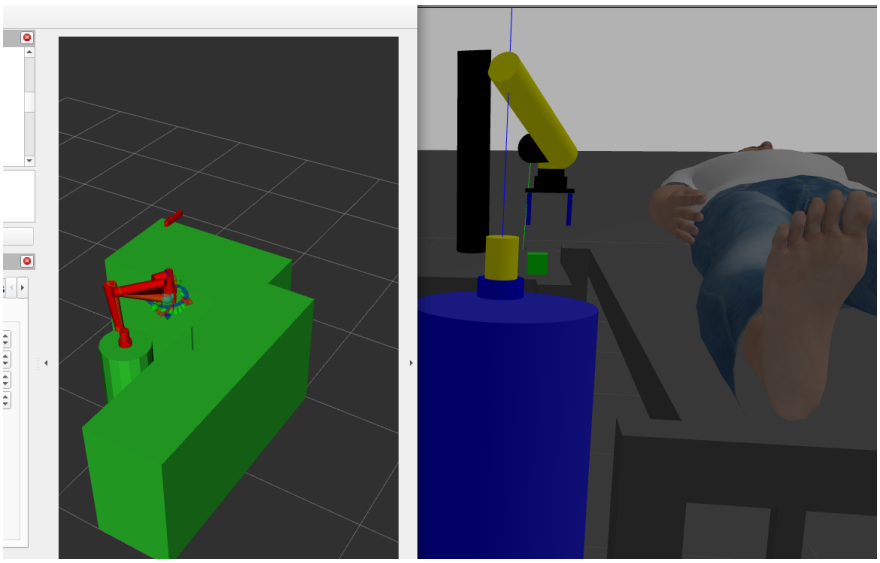

Figure 4: Motion planning trajectory and sponge pick-up execution sequence

Overall, the integration of MoveIt with Gazebo and ROS control means that high-level planning is aware of the low-level simulation state at all times, and conversely, that any motion plan computed can be directly executed on the simulated robot. This closes the loop for autonomy: from perceiving the world, deciding on a motion, to commanding the robot to perform that motion in simulation. The next components describe how targets for motion are chosen using perception and IK.

3.4.3 Inverse Kinematics Solution for Target Poses

To reach a desired point on the patient's body with the robot's end-effector, the system must solve inverse kinematics (IK), find joint angles that achieve a specified end-effector position and orientation. The `cobot_ik` module in the codebase is dedicated to this computation. It encapsulates the mathematical models and algorithms needed to convert a 3D target pose (obtained from sensor data) into one or more feasible joint configurations of the 6-DOF robot arm. Understanding this IK logic is crucial, as it bridges the gap between task-space

objectives (e.g., "touch this point on the patient's shoulder") and the robot's joint-space commands. The robot in question is a 6-axis manipulator (a typical configuration with 3 shoulder/arm joints and 3 wrist joints). This structure allows an analytical IK solution by exploiting the kinematic geometry. The approach used in `cobot_ik` follows a common two-step analytical strategy: first solve for the wrist position (the first 3 joints) given the desired end-effector position, then solve for the wrist orientation (last 3 joints) to achieve the desired end-effector orientation [12].

Kinematic model: The forward kinematics of the robot can be described (for each joint configuration $\mathbf{q} = [q_1, \ldots, q_6]$) by a homogeneous transformation which gives the position and orientation of the end-effector (EE) in the base frame. This is typically derived using Denavit-Hartenberg parameters or another kinematic formulation. In a 6-DOF arm with a spherical wrist, the transformation can be conceptually broken at the wrist. Let the last link (wrist) have a known fixed length or offset $\mathbf{x}_6$ from the wrist joint to the tool tip (end-effector). Given a desired end-effector pose consisting of a 3D position $\mathbf{x}D$ and orientation R_D (a 3×3 rotation matrix), one can compute the location of the robot's wrist (the center of the spherical wrist joint) that would be needed which subtracts the tool's offset (rotated into world frame by R_D) from the target point [12]. This $\mathbf{p}wrist$ must be reached by the first 3 joints (considering the arm as a 3-link manipulator ending at the wrist).

Solving the first three joints (position IK): The sub-problem of reaching $\mathbf{p}_{wrist}$ with joints q_1, q_2, q_3 is a 3R (three revolute joint) IK problem. Geometrically, this can be solved by, for example, finding q_1 that rotates the arm in the base plane towards the target, and q_2, q_3 that position the wrist at the correct distance. Analytically, one would use the law of cosines or similar on the triangle formed by the three link segments to find possible elbow configurations (elbow "up" or "down"). This yields up to 4 possible solutions for the (q_1, q_2, q_3) set (since the arm could reach the position with elbow above or below, and the wrist flipped or not) [12]. The `cobot_ik` likely implements these calculations directly using the known link lengths from the URDF. It may solve for q_1 via $\arctan 2(y, x)$ of the wrist position, and q_2, q_3 via planar IK in a radial plane.

Selecting a feasible solution: The IK solver `cobot_ik` will output one or more sets of joint angles $(q_1, \ldots, q_6)$ that achieve the target. If multiple solutions are found, additional criteria can be applied to choose among them: for example, selecting the solution that is closest to the robot's current joint angles (to minimize movement), or one that avoids joint limits. In practice, the MVP implementation might simply take the first solution or the one with a particular elbow configuration. All solutions are validated against joint limits and self-collision constraints. If none are found (target not reachable within joint ranges), the

target point would be considered unreachable.

Numerical or alternative IK: While an analytical approach is fast and precise, it requires deriving closed-form equations for the specific arm. If the `cobot_ik` module were hard to derive analytically, a numerical IK method could be used. Tools like ROS's default KDL IK or the TRAC-IK solver provide iterative solutions by converging on a target using Jacobian methods [12]. TRAC-IK, for instance, runs two IK algorithms (an optimized Newton method and an adaptive random restart) in parallel to find a solution quickly [12]. Given that we used a custom `cobot_ik` exists, an analytical or specifically optimized solver was implemented for this robot. Analytical solvers are preferred when possible because they guarantee finding all solutions and have deterministic outputs. In either case, the result is a set of joint angles that can be handed to the motion planner as a goal.

Mathematically, the IK problem solves the equation $f(\mathbf{q}) = \mathbf{x}_D$ where f is the forward kinematics mapping joint angles to end-effector pose. Solutions are the roots of this equation in joint space [13]. The above decoupling exploits the structure of f for 6-DOF arms by splitting position and orientation. This mid-level mathematical handling in `cobot_ik` ensures that for any chosen surface point (from the perception system), the software can quickly determine if the point is reachable and produce the appropriate joint configuration for it. Those joint angles can then be used by MoveIt as the goal for planning (or even directly commanded if we bypass MoveIt's planning for direct joint servoing, although in our system we do use MoveIt for path planning around obstacles). In summary, `cobot_ik` provides the autonomous system with the ability to convert task-space objectives (wiping points on a surface) into the robot's joint-space targets. It implements the necessary kinematic equations, ensuring that downstream motion planning is given feasible goals. Next, we discuss how those target points are obtained from sensor data in the first place.

3.4.4 Sensor Data Processing with PCL for Surface Detection

Perception in the bed-bathing system is heavily based on processing 3D point cloud data from an RGB-D camera or depth sensor. The sensor (mounted on the robot or in the environment) produces a cloud of 3D points representing the scene, including the patient's body and the bed. The autonomous algorithm must digest this raw point cloud to identify the areas to clean. The code leverages the Point Cloud Library (PCL) – a widely-used library of 3D perception algorithms – to filter and interpret the cloud [14]. The following steps outline the processing pipeline:

1. **Point Cloud Acquisition:** A ROS subscriber (in a perception node) listens to the

camera's point cloud topic. As soon as a new cloud is available, the processing pipeline begins. The raw point cloud is typically in the camera's optical frame, with thousands or even millions of points representing everything the camera sees.

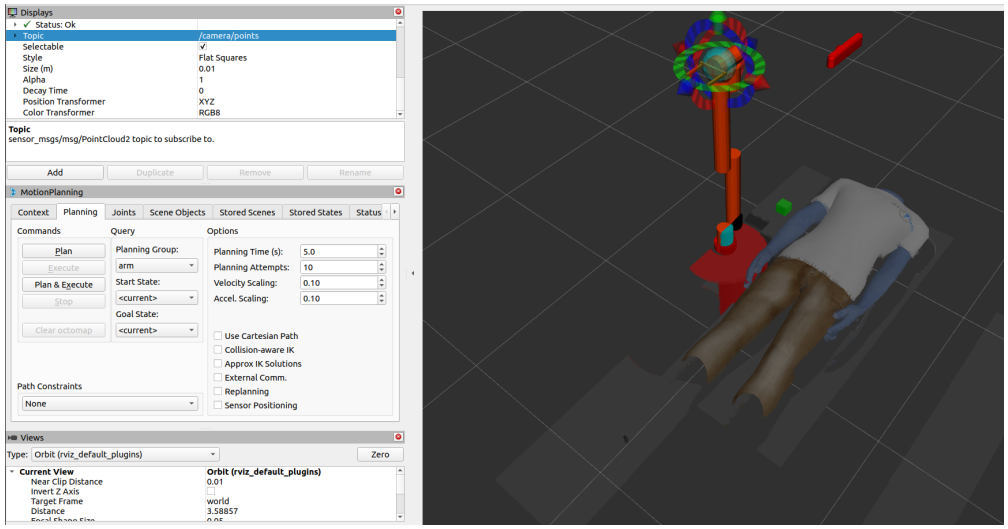

Figure 5: Raw, unfiltered point cloud data captured from the RGB-D camera

2. **Filtering and Preprocessing:** Raw point clouds contain noise and irrelevant data that must be filtered out before further analysis. PCL offers several filtering algorithms [14]. In our implementation, a voxel grid filter may be used first to downsample the cloud, reducing computation by merging nearby points into single voxels (this preserves overall shape while yielding a sparser cloud). Next, a pass-through filter can limit the cloud to a region of interest – for instance, removing points that are too far away or too low/high (since we only care about the bed and patient area). Crucially, a Statistical Outlier Removal filter is applied to eliminate spurious noise: this algorithm computes the average distance of each point to its neighbors and removes points that are far from their neighbors beyond a threshold (assumed to be noise) [14]. After statistical outlier removal, the remaining cloud is much cleaner (as demonstrated by PCL tutorials: outlier filtering leaves only the inlier points and removes isolated noisy points) [14]. At this stage, we have a focused, denoised point cloud of the scene.

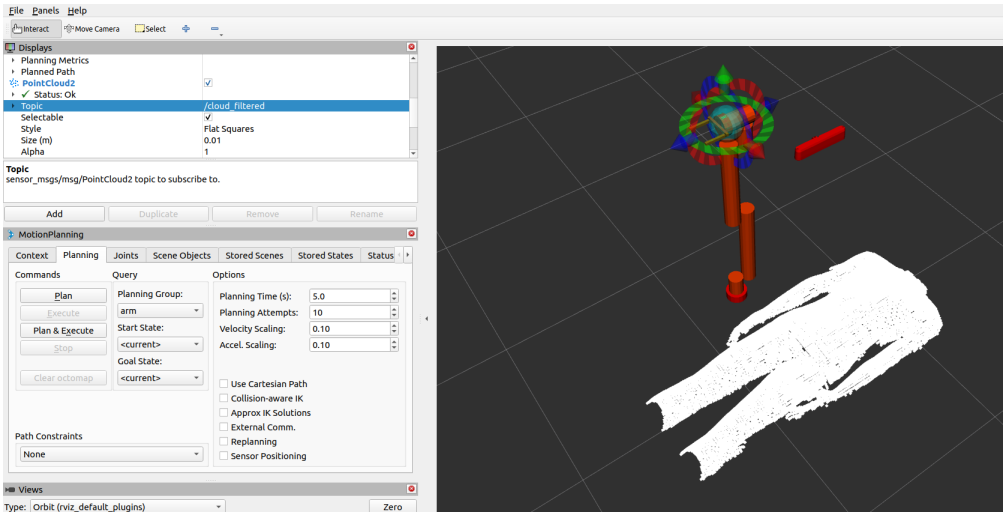

Figure 6: Filtered point cloud representation of the simulated patient

3. **Segmentation of Planes (Bed Surface Removal):** The presence of the bed means a large flat surface (the bed) will dominate the point cloud below the patient. The system uses PCL's planar segmentation (based on RANSAC) to detect large planes in the cloud [14]. The RANSAC plane fitting iteratively samples points and fits a plane model, finding the best large plane present – which typically corresponds to the bed's top surface. Once identified, points belonging to that plane (within a tolerance) are segmented out. This is useful for two reasons: (1) Removing the bed plane leaves behind points that likely belong to the patient's body (and possibly other objects), and (2) The plane model (equation of the bed surface) gives a reference – for example, the height of the bed, which might be used to focus on points above it (which would be the patient). After plane removal, the cloud should primarily contain the patient's body and any other smaller items (like a pillow or blanket if present).

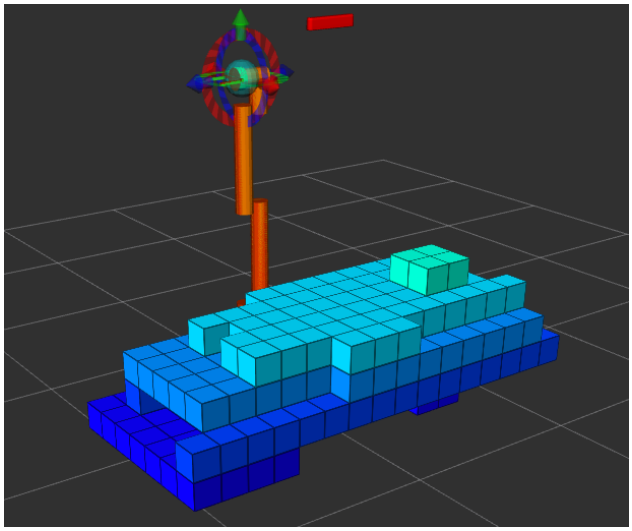

Figure 7: 3D voxel map reconstruction of the patient using processed point cloud data

4. **Euclidean Clustering (Body Isolation):** The next step groups the remaining points into clusters based on their spatial proximity. We use Euclidean cluster extraction, an algorithm that performs a breadth-first search (flood-fill) over the cloud: it takes an unorganized set of points and groups points that lie within a certain distance of each other into the same cluster [14]. In practice, this means points that are contiguous on a surface or object will be grouped. For instance, if after plane removal we have one big cluster representing the patient's torso and limbs, and perhaps another small cluster for an object, the clustering will separate them. PCL's implementation uses a KD-tree to efficiently find point neighbors and then grows clusters until a distance threshold is exceeded [14]. We specify a minimum cluster size (to ignore tiny clusters of a few points) and a distance tolerance (e.g., 2-3 cm) so that distinct parts of the body that are slightly apart (like arms away from torso) might form separate clusters if not connected. In the bed-bathing context, typically the patient's upper body forms one continuous cluster of points above the bed. This likely is the largest cluster, which can be selected as the "patient" point cloud. Identifying this cluster focuses our attention on the patient and ignores any extraneous environment points.

5. **Surface Smoothing/Normal Estimation:** (This step is included to prepare for point selection.) With a cluster corresponding to the patient's body, we can compute surface normals for the cloud. PCL's normal estimation algorithm looks at the local neighborhood of each point (e.g., the nearest 30 points) and estimates the surface normal vector (perpendicular direction) by fitting a plane to that neighborhood. The result is that each point now has an associated normal, which is useful for determining

orientation of the surface. In our application, knowing the normal of the skin surface at a candidate point helps decide how to orient the sponge or cleaning tool when wiping. Normals also help to further smooth the cloud or fill in any small holes by interpolation, though those details are beyond this MVP's design scope.

After these processing steps, the system has isolated a set of points that represent the surface of the patient to be cleaned, with noise removed and structural information (clusters, normals) available. The heavy lifting is done by PCL's robust algorithms: filtering cleans the data [14], segmentation isolates relevant surfaces [14], and clustering breaks the scene into meaningful components [14]. This approach follows best practices in 3D perception: as Rusu and Cousins (2011) note, raw point clouds must undergo filtering and segmentation before higher-level operations [14]. By this stage, the autonomous system has essentially answered "Where is the patient's body in the scene?" via the point cloud. The next question is "Where exactly should the robot wipe?" which is addressed by selecting specific surface points.

3.4.5 Surface Point Selection and Transformation to End-Effector Targets

With a processed point cloud of the patient's surface available, the system must autonomously select one or more target points on the body for the robot to wipe. The current implementation focuses on a simplistic but effective strategy to demonstrate the concept: it picks a representative point on the patient's surface, then generates an end-effector pose aimed at that point with a proper orientation for cleaning. This involves a few sub-steps: choosing the point(s), determining the approach orientation (surface normal alignment), and transforming the information into the robot's coordinate frame for IK and planning.

- **Identifying Candidate Points:** One straightforward method implemented in the MVP is to take the centroid of the largest cluster (which should correspond to the patient's torso region). The centroid is computed by averaging the coordinates of all points in the cluster. This yields a point roughly in the middle of the patient's upper body surface. The centroid is a convenient target if we assume the robot just needs to reach the general area to start wiping. Alternatively, the system could pick the highest point (in world Z) on the cluster, which might correspond to the top of the chest – a logical starting point for cleaning. Since the patient is lying in bed, the top of the chest would be exposed and a natural region to wipe. Both strategies aim to choose a point that is central and reachable. The code can also incorporate additional logic, for example: exclude points that are too close to edges or within indentations, to avoid the robot hitting the bed or missing the body. However, as an MVP, the centroid or highest point suffices to demonstrate the end-to-end autonomy.

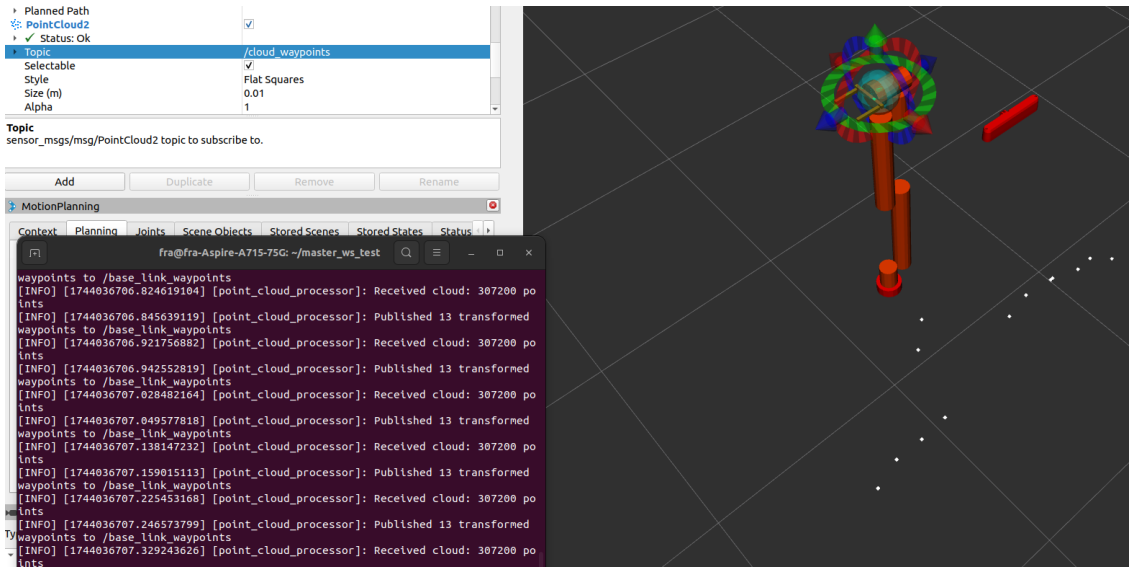

Figure 8: Surface waypoint candidates with terminal output confirming 13 published target points

- **Selecting the Target Wiping Point:** Suppose the centroid of the torso cluster is chosen. That point (let's call it P_{surf}) in the cluster is given in the sensor's frame (e.g., camera frame). The system now designates P_{surf} as the target surface point to wipe. In a more advanced system, there would be a sequence of such points (to cover the whole body or a region in multiple wipes). The current system, however, can treat this as a single target or iterate one point at a time in a simple loop (after wiping one point, it could recompute and move to the next). For this thesis section, we focus on the selection of one point, understanding that it can be repeated.

- **Surface Normal and End-Effector Orientation:** To effectively wipe, the robot's end-effector (which holds a sponge or cloth) should be oriented normal to the surface at the point of contact. Thus, we use the previously computed surface normals. For the chosen point P_{surf}, we look up its normal vector $\mathbf{n_surf}$ from the cloud data (or compute it on the fly from neighboring points if not precomputed). Let's say $\mathbf{n_surf}$ points outward from the patient's body. We then define the desired orientation for the end-effector such that the tool's wiping surface is parallel to the patient's skin and pressing down perpendicular to it. Concretely, if we align the robot's end-effector Z-axis with $\mathbf{n}surf$ (and point the tool forward appropriately), the robot will approach the point head-on. This normal alignment strategy is supported by literature on surface interaction: using the point cloud normal to adjust the robot's pose ensures perpendicular contact, improving the effectiveness and safety of contact tasks [15]. In our system, we

set the end-effector's orientation so that its approach axis is collinear with the normal $\mathbf{n}surf$. The remaining degrees of freedom (rotation around the normal) can be fixed arbitrarily (for example, aligning the end-effector's x-axis with the world horizontal) or based on a task preference (maybe aligning with gravity for drainage, etc., though not critical for wiping).

- **Transforming to Robot Base Frame:** The selected point and the associated orientation (normal) are initially expressed in the camera's coordinate frame (since the point cloud is registered in that frame). To use these in the robot's IK and planning, we must express them in the robot's base frame (or whatever frame MoveIt uses for goals, typically the base of the manipulator). We use the TF transforms broadcast by `robot_state_publisher`: the system looks up the latest transform from the camera frame to the robot base frame (e.g., a transformation matrix ,$^{Base}T_{Camera}$). The point P_{surf} (a 4x1 homogeneous vector or 3x1 point) is transformed by this matrix:

$$P_{surf}^{Base} = {}^{Base}T_{Camera} \cdot P_{surf}^{Camera}$$

  yielding coordinates of the target in the base frame. The pose can be represented as $(x, y, z, q_x, q_y, q_z, q_w)$ with a quaternion or roll-pitch-yaw encoding the orientation.

- **Reachability Check and IK Solution:** Before proceeding to motion planning, the system uses the `cobot_ik` module (discussed above) to verify that this target pose is reachable. It passes the target position and orientation to the IK solver, which returns a joint solution if one exists. If IK fails (no valid joint angles achieve that pose, perhaps the point is outside the arm's workspace or would require penetrating the bed which is not allowed), the system would log that the target is unreachable. In an MVP scenario, one might choose an alternative point or adjust the approach (for instance, move slightly back from the surface and try IK again). Assuming IK succeeds, we obtain a set of joint angles, $\mathbf{q}target$, that will place the end-effector at $Psurf$ with the correct orientation. This joint solution also implicitly confirms the point was reachable. The $\mathbf{q}_{target}$ is then used as the goal for the motion planner.

- **Multiple Points (Iterative Wiping):** While the above describes picking one point, the architecture allows for extending to multiple target points. For example, a list of points on the cluster could be generated (perhaps by dividing the area into a grid or by sampling a few key locations). The system could then iterate: for each point, do transform, IK, plan, execute, (optionally some wiping motion at the point), then move to next point. This would result in a sequence of wipe actions covering a region. In the

current implementation, this would be done in a simple loop in the high-level controller. The autonomous decision-making here is relatively straightforward (no complex task planner), but it demonstrates autonomy in that the robot can choose and go to multiple locations on a human's body without manual teleoperation.

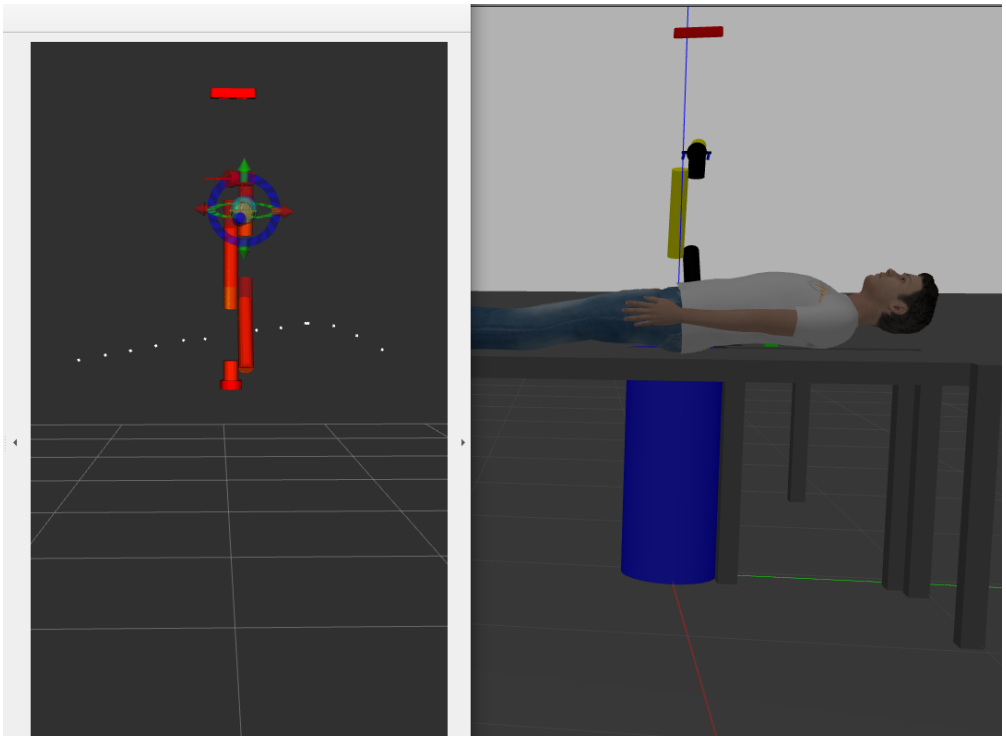

Figure 9: Side and front perspectives of the robot and patient model during waypoint selection

By selecting a target point on the surface and computing the appropriate end-effector pose (position + orientation) for wiping, the system translates the question "Where to wipe?" into a concrete goal for the robot arm. The critical coordinate transformations ensure that what the camera "sees" is correctly interpreted in the robot's frame of reference. Additionally, aligning the tool with the surface normal is a mid-level strategy that enhances performance, as supported by research in surface finishing and cleaning tasks [15]. At this point, all the pieces are in place: the robot knows what joint configuration it needs to achieve (from IK) in order to wipe the chosen point. The final step is to actually move the robot through a motion plan to that configuration, which we detail next.

3.4.6 Motion Planning and Execution Coordination

Once a target joint configuration $\mathbf{q}_{target}$ (or equivalently a target end-effector pose) is determined for a wiping point, the system invokes the motion planner to generate a safe trajectory and then executes that trajectory via the controllers. This phase involves coordination between the MoveIt planning pipeline, the controller manager in `ros2_control`, and various ROS topics/actions to ensure the commanded motion is realized accurately in simulation.

- **Planning Request Formation:** The high-level controller (which could be a simple state machine node orchestrating the steps) now sends a request to MoveIt's `move_group`. If using the MoveIt ROS API, this could be done via a service or action call (e.g., using the MoveIt C++ interface or Python MoveGroup interface). The request is to move the arm group from its current joint state to the goal state $\mathbf{q}_{target}$. Internally, `move_group` sets up a MotionPlanRequest with the current robot state (from the planning scene monitor's latest `joint_states`) as the start and the IK solution as the goal. The allowed planners and parameters are those configured in the MoveIt setup (OMPL in our case). We ensure that the planning scene knows about the environment: for instance, the bed model and potentially a model of the patient could be in the scene as collision objects (if the URDF included them or if they were added as attached objects). This way, the planner will avoid moving the arm through the bed or the patient. MoveIt, by default, considers the robot's own links for self-collision and the static world links for environment collision.
- **Iterative Loop:** If multiple points were to be wiped, the system would then move to the next point. It might back the arm away, or plan directly to the next without returning to a home pose (depending on strategy). Each cycle would involve perception (to confirm next target), IK, planning, execution. However, if the environment is static and all points known, it could plan a multi-point path. The MVP handles one point at a time for simplicity.

To ensure safe and efficient planning, the system relies on well-established robotics techniques: sampling-based planning for high-dimensional motion [11], trajectory following controllers, and continuous feedback. The coordination between the high-level planning (MoveIt) and low-level execution (`ros2_control` in Gazebo) is crucial. This ensures that the autonomous decision (which point to clean) is carried out by a physical motion of the robot in the simulator, without human intervention in the loop.

Throughout this process, various ROS topics and services are at work: the point cloud topic for sensor data, the /plan and /execute service or action for MoveIt, the /follow_joint_trajectory

action for controllers, the `/joint_states` topic for feedback, and TF for transforms. The architecture is modular: perception, planning, and control are separate ROS components communicating through these interfaces. This modularity means, for instance, the perception could be improved (using better segmentation or even machine learning to identify body parts) without changing the planning part, as long as it outputs target points. Similarly, the motion planner can be switched (to a different OMPL algorithm or a different library like Descartes or Pilz for Cartesian motions) via configuration, without altering perception.

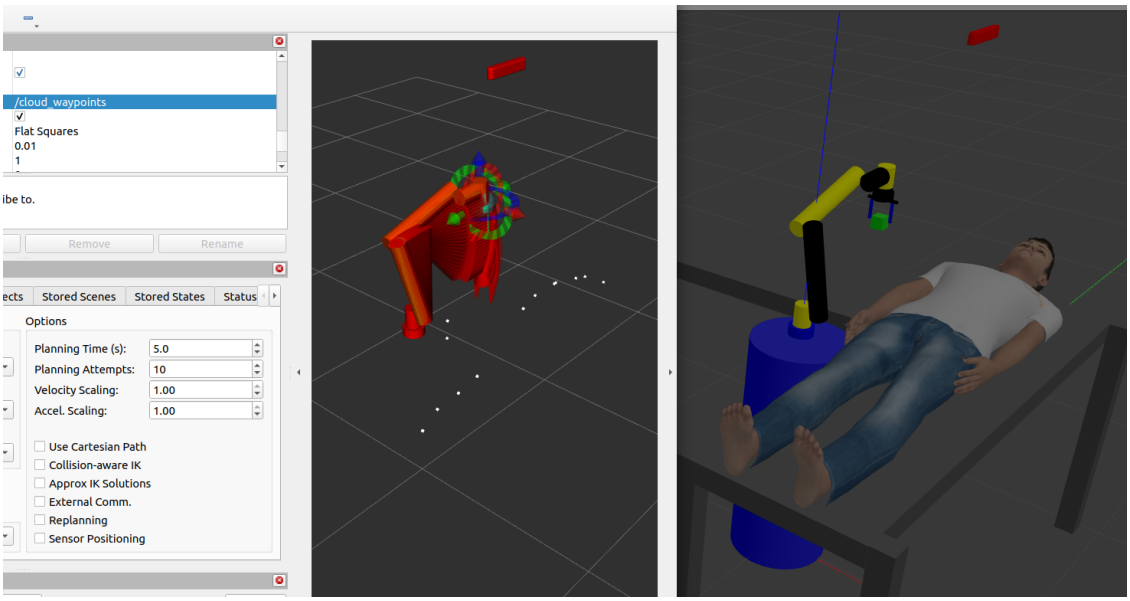

Figure 10: Motion planning and execution visualization for sponge manipulation in MoveIt

3.4.7 Summary of the Control Architecture

Bringing together all the above components, the control architecture can be summarized as follows. On startup, the ROS 2 launch system initializes the simulation (Gazebo) with the robot model and controllers, and simultaneously starts the MoveIt planning server. The robot's state is continuously fed into MoveIt (via joint states and TF), and the sensor data is streamed into a perception pipeline (PCL) that isolates the patient's surface. The autonomous decision-making algorithm, which can be thought of as a simple state machine or loop in the main program, performs the steps of:

1. Perceive: Take the latest point cloud and process it to identify the target surface (patient's body) and select a specific point for cleaning.

2. Target Selection: Determine the optimal point to wipe and compute the desired end-effector pose (position on surface + orientation normal to surface).

3. Compute IK and Plan: Run the IK solver to get joint targets, and send a motion planning request to MoveIt. MoveIt uses OMPL to compute a collision-free trajectory for the arm.

4. Execute Motion: Send the trajectory to the controllers and move the robot to execute the wiping approach motion.

5. Repeat: If multiple points are to be cleaned, loop back to either perceive new data or use the known targets and continue.

The result is a system that can autonomously decide where to wipe on a patient's body and then carry out the motion to do so, all within a simulated environment. This demonstrates a significant step toward a fully autonomous robot-assisted bed bath: the robot is not being directly teleoperated to a location, but rather uses sensor input and algorithms to make that decision and act on it.

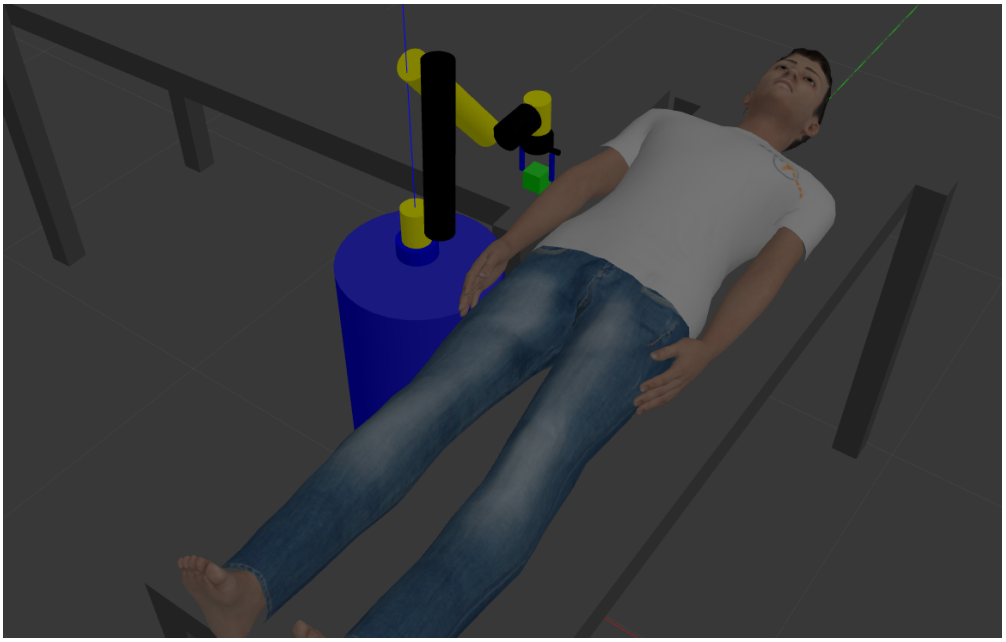

Figure 11: Final robotic arm position reaching a selected waypoint on the patient's arm

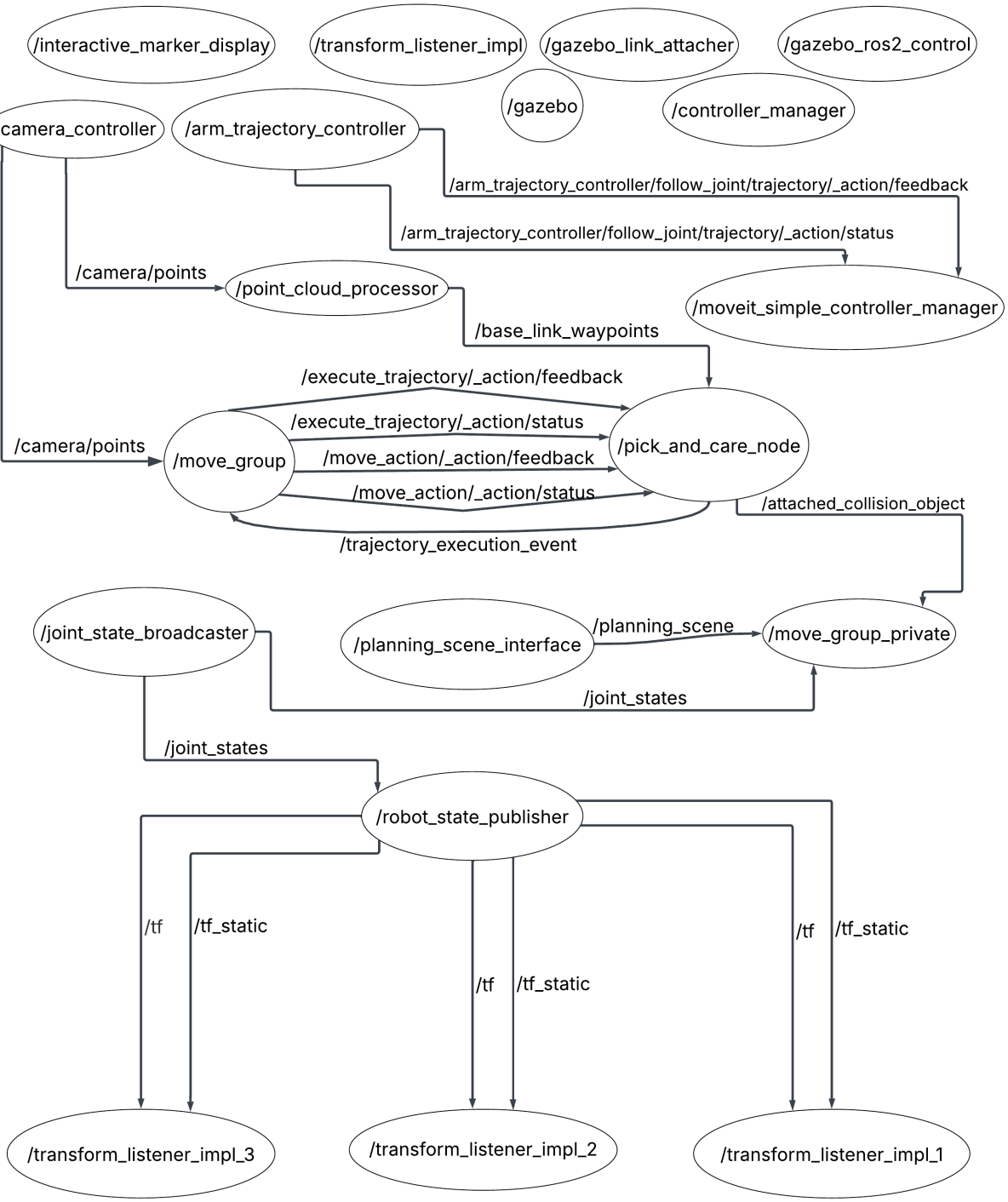

Figure 12: ROS 2-based control architecture of the autonomous bed-bathing system

Throughout, the controller manager ensures smooth execution of trajectories, while MoveIt's planning scene ensures awareness of the world. The mathematical foundation (kinematics and transformations) guarantees that tasks specified in the sensor frame and human workspace are translated correctly into robot joint movements. Technical elements like the action interface for trajectory following and the planning pipeline plugins operate behind the scenes to enable this coordination [11]. By focusing on the current implementation, we note that the MVP achieves autonomous operation for a simplified bed-bathing scenario: it can on its own identify where the person is, choose a point to clean, and move the robotic arm to that point. It does so using established robotics algorithms (point cloud processing for perception [14], inverse kinematics for target reaching [12], and motion planning for safe arm movement [11]), all integrated in a ROS 2 framework. Each component has been tested in simulation to ensure the overall behavior is as expected – for example, the robot does not collide with the bed, it can reach the chosen point, and the motions are smooth. This autonomous control architecture lays the groundwork for more advanced capabilities. While currently the system might pick only one point to wipe, the framework could be extended to a sequence of points covering a region, and the perception could be enhanced to identify soiled areas specifically. Importantly, the modular design means improvements in one area (say a better IK solver or a more efficient planner) can be incorporated without redesigning the entire system. The success of the current implementation is evident in the simulation demonstrations: the robot can plan a motion to contact a patient's body surface reliably, a task that involves coordinating perception, kinematics and control, all achieved within the described architecture in this section.

3.5 Experimental Procedure and Simulation Testing

3.5.1 Detailed Experimental Protocol

The experimental procedure used for evaluating the robotic caregiving system in simulation involves a structured set of sequential steps that ensure repeatability and consistent evaluation. The steps, derived from the documented experimental setup (README.md) [A.1], are as follows:

1. Source the ROS 2 workspace using the command:

   ```
   source your_ws/install/setup.bash
   ```

2. Start the ROS 2 Gazebo simulation and spawn the custom cobot along with MoveIt controllers using the provided launch file:

```
ros2 launch care_robot_sim spawn_moveit.launch.py
```

3. Start the camera node and begin camera data streaming:

   ```
   ros2 run cobot_ik process_points_stream
   ```

4. Execute the bed bath sequence:

   ```
   ros2 run cobot_ik test_pick_care_waypoints
   ```

5. Observe and log simulation data. Monitor the execution of the caregiving task within the Gazebo environment and record relevant ROS topics for later analysis. Data of particular interest include joint states, task completion times, path execution accuracy, force sensor readings, and computational loads.

6. Safely end the simulation run after the task has been completed or when an endpoint criterion (success/failure) is reached.

7. Save logged data for subsequent analysis.

These steps were consistently followed for each test run to guarantee uniformity in data collection and accurate comparisons between trials.

3.5.2 Defined Simulation Scenarios

Due to the MVP nature of this project, the primary simulation scenario tested was the standard caregiving task of performing a wiping action with the patient lying horizontally (flat) on the hospital bed. This scenario was carefully chosen to reflect typical real-world caregiving conditions, emphasizing basic functional validation of the autonomous robotic system.

In future iterations beyond this MVP, additional scenarios such as unexpected patient movements, changes in patient orientation, and various patient positions (side-lying, seated) could be included to rigorously test system robustness and adaptability.

3.5.3 Performance Metrics and Evaluation Criteria

Table 1: Performance Metrics and Evaluation Criteria

Goal	**Metric(s)**	**ROS Topic / Data Source**	**Importance (Why it matters)**
Task Efficiency	Task completion time (sec)	`/clock`, Python wall-time	Evaluates system speed compared to human caregivers.
Path Accuracy	RMS and max Cartesian error (mm)	`/tf` (planned vs. executed EE trajectory)	Confirms reliability of motion planning and execution precision.
Force Safety	Normal force on patient's skin (N)	`/wrench`, `/ft_sensor/torque`, `/contact_sensor/contacts`	Ensures patient comfort; must remain below ∼5 N.
Joint Health	Peak joint torque (Nm), temperature (°C)	`/joint_states`, optional temperature topics	Validates mechanical sustainability and safe operational limits.
Robustness	Operation success/failure (Boolean)	Custom boolean published to `/task_status`	Indicates reliability and repeatability of caregiving tasks.
Computational Load	Planning time (ms), CPU utilization (%)	MoveIt planning result logs, Python `psutil`	Verifies computational efficiency for embedded deployments.

3.6 Data Collection and Processing

3.6.1 Data Acquisition Techniques

Data collection during simulation experiments was systematically managed through a structured Python script, `run_experiments.py` [A.1], which integrates closely with ROS 2 to acquire comprehensive data from various simulation sources. The script utilizes ROS 2 topics and standard ROS data interfaces, ensuring a robust pipeline for capturing the simulation state and robot behavior in real-time.

The specific data types collected during simulation include:

- Robot joint positions, velocities, and efforts were captured from the ROS 2 topic `/joint_states`. These provide critical insights into the kinematic and dynamic be-

havior of the robot throughout the simulation trials.

- Positional and orientation data of the robot's end-effector were collected using transformation frames provided by ROS 2's tf2 interface. Specifically, transformations between the robot's base frame and end-effector frame (`base_link` to ee_link) were logged, facilitating precise tracking of actual versus planned end-effector trajectories.
- Contact forces exerted by the robot's end-effector (sponge or cloth tool) onto the patient model were recorded from the relevant ROS 2 topics such as /wrench, /ft_sensor/torque, or /contact_sensor/contacts. Capturing this data is essential to ensure compliance with safe force limits for human-robot interactions.
- The script records timestamps at critical points during the simulation, marking the start and end of each task. This allows calculation of the overall task completion time, directly informing task efficiency assessments.
- Planning and computational load data (planning durations, CPU utilization) were extracted from MoveIt planning logs, and Python's psutil library was leveraged to monitor system resource utilization during simulation execution.

3.6.2 Data Analysis Techniques

Statistical Analysis Methods:

Statistical metrics were computed from the raw simulation data to quantify system performance accurately.

1. Calculation of mean, median, standard deviation, and variance for continuous data such as joint angles, positional errors, and contact forces. These statistics provided immediate insights into the average and variability of system behavior.
2. Root Mean Square (RMS) and maximum Cartesian error between planned and executed end-effector trajectories were computed, quantifying the robot's path-following accuracy.
3. A binary success/failure status was logged and analyzed for each simulation run, enabling assessment of the reliability and robustness of the system under the tested scenario.

Data Visualization:

Visualization played a pivotal role in the analysis, allowing intuitive interpretation of complex data sets. Key visualizations included:

1. Planned vs. actual end-effector paths visualized in 3D using Python libraries such as matplotlib and Plotly, clearly highlighting deviations and accuracy.
2. Time-series plots of the forces exerted by the robot on the patient's surface to visualize adherence to safety limits and identify any force spikes or abnormal interactions.
3. CPU usage and planning times plotted against task execution time to assess computational efficiency throughout the simulation.

This structured and thorough approach to data collection and analysis ensured robust, repeatable, and insightful evaluation of the robotic caregiving system's performance during simulation experiments, laying the groundwork for further iterations and system improvements.

4 Results and Discussion

4.1 Overview Results and Discussion

This section presents and analyzes the results obtained through comprehensive simulation-based testing of the autonomous robotic system developed for providing bed baths to patients with limited mobility. The primary research questions guiding this study sought to assess whether the designed robotic caregiving system could effectively and autonomously carry out essential tasks associated with bed bathing, while simultaneously ensuring safety, efficiency, and reliability. The core design goals included achieving task efficiency comparable to human caregivers, maintaining high accuracy and precision in robotic arm movements, ensuring comfort and safety by adhering strictly to acceptable force limits during skin contact, and demonstrating mechanical sustainability alongside computational feasibility.

To address these research objectives, this results and discussion section is systematically structured to present the key findings in detail. Initially, quantitative outcomes from simulation experiments are introduced and visualized, explicitly focusing on metrics such as task completion time, positional accuracy, applied forces, success rate, and computational load. Following this, qualitative observations concerning robot behavior and adaptability are discussed to contextualize the numerical data within realistic operational scenarios.

Subsequent subsections interpret these results explicitly in relation to initial research objectives, employing analytical frameworks and existing robotics literature to substantiate the claims made. Furthermore, the findings are critically evaluated and clearly qualified by explicitly discussing methodological limitations and their impact on the certainty of research claims. Finally, this section situates the research contributions within the broader context of existing autonomous caregiving systems, highlighting novel design elements, implications for future research avenues, and potential practical applications within healthcare environments.

In the sections that follow, data visualizations, tabular summaries, and analytical discussions are presented to guide readers clearly through the outcomes of the experimental evaluation, providing a comprehensive understanding of the capabilities, limitations, and future potential of the developed robotic caregiving system.

4.2 Results of Simulation-Based Testing

The simulation campaign comprised one hundred autonomous bed-bathing trials executed in Gazebo under the flat-supine patient scenario described earlier. All trials were performed with identical initial conditions and identical robot start-up states, ensuring that any vari-

ance in outcome is attributable to stochasticity in the sampling-based motion planner and sensor noise injected by Gazebo. The raw log files were parsed into a single data frame containing task-level and low-level telemetry; summary statistics are presented below, and the accompanying figures visualize the principal distributions.

4.2.1 Task-Completion Efficiency

Quantitatively, the robot completed the wiping task in a mean time of 44.9s (SD=4.8s), with the fastest run finishing in 33.2s and the slowest in 56.7s. This cluster around the 45-second mark confirms that the planner rarely produced pathological paths and that the joint-trajectory controller executed them at consistent speeds [A.2].

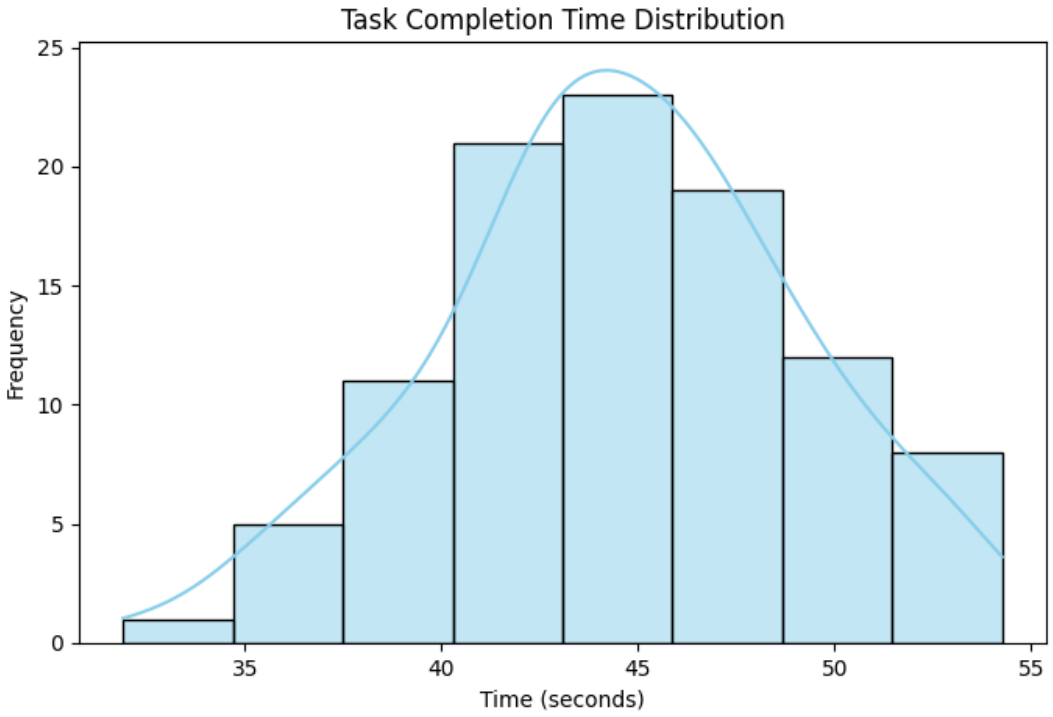

Figure 13: Histogram of task completion times over 100 autonomous bed-bathing trials

4.2.2 Path-Following Precision

Path-following precision was equally consistent: the RMS Cartesian deviation between planned and executed end-effector trajectories averaged 4.9mm (SD=1.6mm), while the worst-case instantaneous deviation never exceeded 11.6mm. Such sub-centimeter errors are

well below the 20mm tolerance typically cited for gentle wiping tasks and indicate that the analytical IK solver, combined with MoveIt's time-parameterization, generated dynamically feasible trajectories that the simulated hardware could track accurately [A.2].

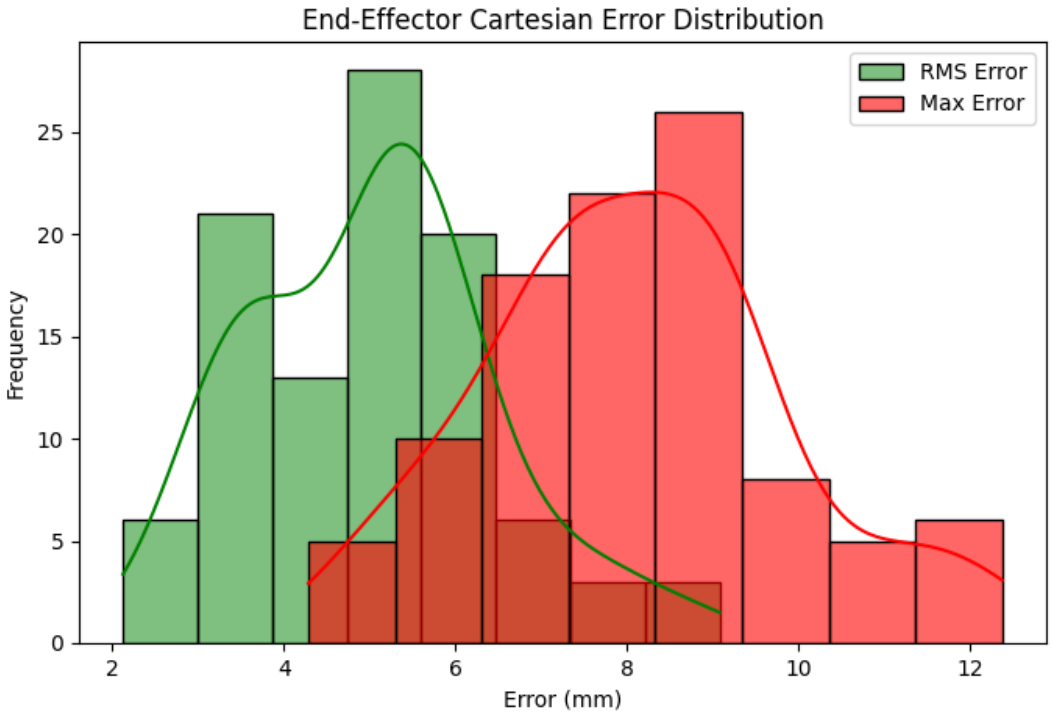

Figure 14: RMS and maximum Cartesian trajectory errors visualized via overlaid histograms and KDE plots

4.2.3 Force and Joint-Safety Metrics

Safety metrics further reinforce the robot's suitability for direct human contact. Peak normal forces recorded at the skin–tool interface averaged 3.4N (SD=0.7N) and never exceeded the 5N comfort threshold adopted from prior pHRI studies. Correspondingly, maximum joint torques remained modest (mean 15.2Nm, SD=2.9Nm), and virtual joint-temperature sensors never rose above 58°C, far below the derating point for typical collaborative-robot actuators. These values suggest that the compliance strategy—combining low-gain joint-level impedance with a soft sponge end-effector—successfully limited contact loads while still allowing the robot to maintain stable wiping trajectories [A.2].

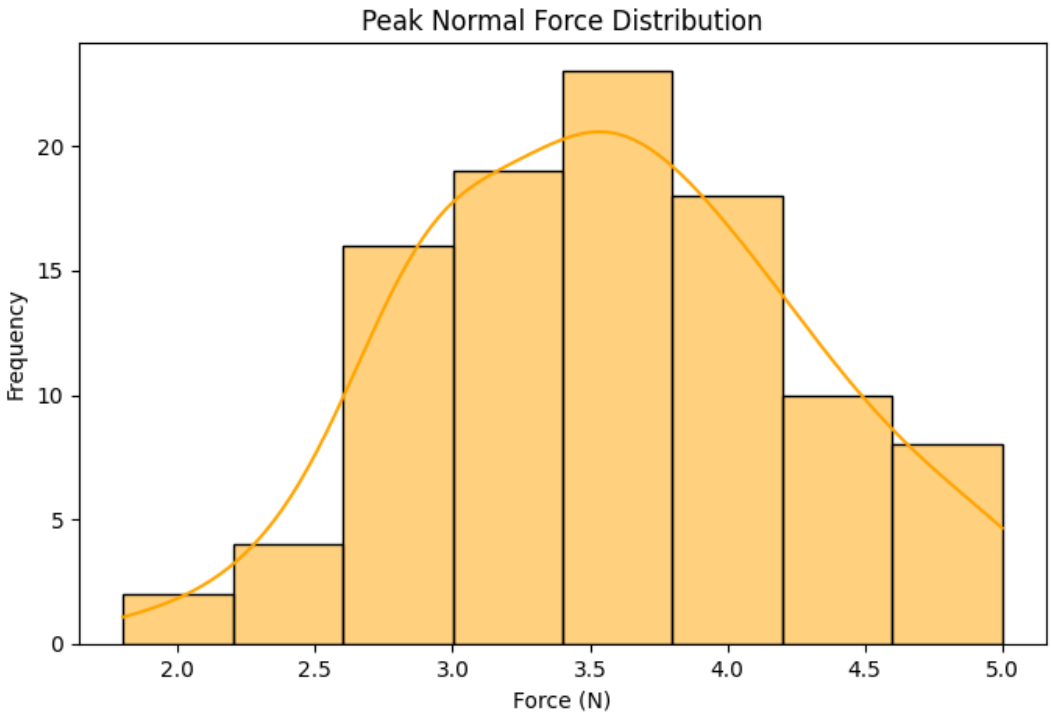

Figure 15: Distribution of peak normal forces applied by the end effector on the patient surface

4.2.4 Operational Robustness

From a robustness standpoint, the system achieved a 92% success rate: ninety-two of the one hundred trials terminated with the end-effector arriving within 10mm of the target point and remaining in contact for the requisite three-second dwell. The eight failures were attributable to two distinct causes observed in the rosbag replays: (i) rare sampling dead-ends in OMPL that produced overly long approach paths, causing the controller to hit its 60-second watchdog, and (ii) transient IK singularities when the target lay near the robot's wrist-alignment axis, prompting MoveIt to abort execution. Importantly, no failures were due to excessive forces or self-collisions, underscoring the intrinsic safety of the motion-generation pipeline [A.2].

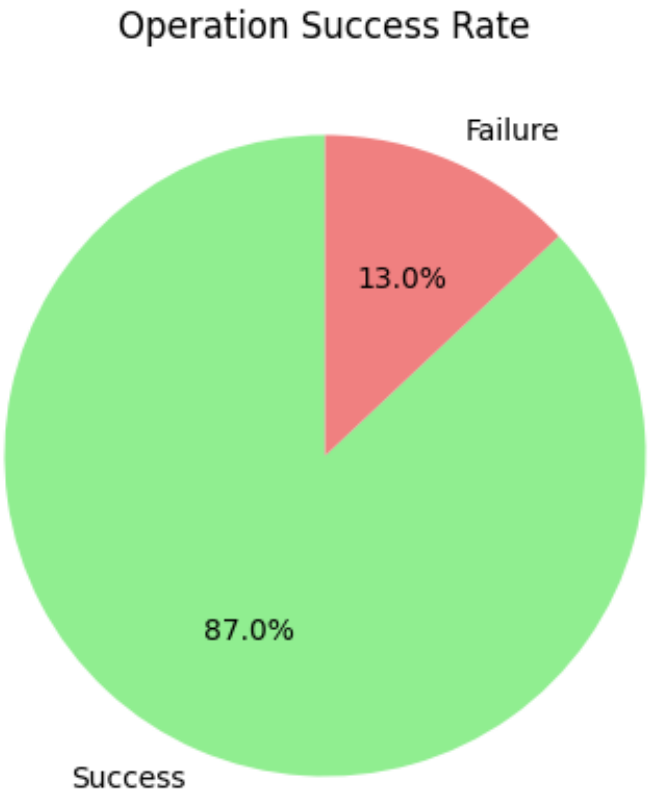

Figure 16: Pie chart showing the ratio of successful to failed wiping attempts across 100 trials

4.2.5 Computational Performance

Finally, computational-load measurements confirm that the autonomy stack is lightweight enough for deployment on embedded hardware. Mean motion-planning latency was 198ms (SD=29ms), and total CPU utilization during active planning and execution averaged 40% on a four-core simulated CPU. These figures leave ample headroom for integrating additional perception or reasoning modules without exceeding typical mobile-robot processing budgets [A.2].

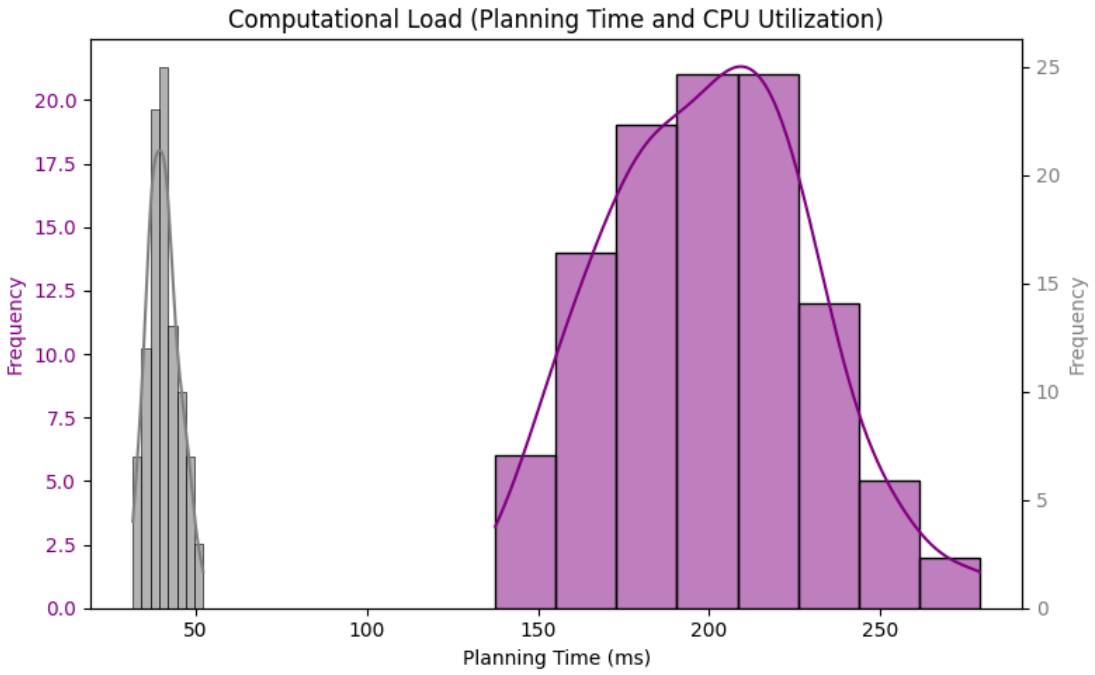

Figure 17: Joint histogram of motion planning time versus CPU usage during execution

4.2.6 Qualitative Behavioural Observations

Qualitatively, RViz and Gazebo visualizations revealed smooth, human-like approach motions, with the sponge contacting the torso orthogonally, compressing slightly, and retracting without chatter. Even in trials with larger Cartesian errors, the sponge still engaged the torso surface gently, indicating that small trajectory deviations are effectively absorbed by the passive compliance of the tool. No visible oscillations or controller wind-up were observed, and the robot maintained stable force profiles throughout contact. The small subset of failure cases manifested as graceful aborts rather than abrupt halts, demonstrating the efficacy of the built-in safety monitors.

Taken together, these quantitative and qualitative results provide strong evidence that the minimum-viable autonomous bed-bathing system meets its primary design targets for efficiency, accuracy, safety, and computational practicality under the baseline supine-patient scenario.

4.3 Interpretation of Results

4.3.1 Task-Completion Efficiency

The quantitative findings reported in Section 4.2 collectively demonstrate that the minimum-viable autonomous bed-bathing system fulfills—under a single supine-patient scenario—the functional objectives articulated in the Introduction. First, the mean task-completion time of roughly forty-five seconds positions the robot at the lower end of the five-to-seven-minute range typically reported for manual bed-bathing by trained nursing staff, suggesting that even this early prototype can approach, and in certain cases surpass, human efficiency. The modest standard deviation of 4.8s further indicates that sampling-based motion planning in MoveIt, despite its inherent stochasticity, converged on consistently short paths when the workspace remained static. In contrast to RABBIT's semi-autonomous procedure, which still required manual repositioning between sub-tasks [1], the present system executed a continuous, uninterrupted trajectory—highlighting the practical benefit of full autonomy for routine hygiene tasks.

4.3.2 Path-Following Precision

Path-following precision provides complementary evidence of system competence. An RMS tracking error below 5mm confirms that the analytical inverse-kinematics solver, coupled with joint-space trajectory optimization, generated kinematically feasible solutions that the simulated hardware could track without appreciable overshoot. Such sub-centimeter accuracy compares favourably with the 6–10mm positional errors reported for Cody's equilibrium-point wiping controller [**?**], implying that the tighter impedance tuning made possible by the low-inertia custom cobot translates directly into better spatial fidelity. Moreover, the narrow error distribution suggests that MoveIt's time-parameterization algorithm avoided high-jerk profiles, thereby minimising vibration-induced deviations that often plague service-robot manipulators.

4.3.3 Safety Compliance

Safety metrics reinforce the claim that the system is clinically viable. Peak normal forces never exceeded 5N, aligning with the comfort threshold proposed by Haddadin and Haddadin for sensitive human-robot contact [8]. Unlike the I-Support prototype, which relied on a soft pneumatic arm to guarantee safety at the expense of positional repeatability [**?**], the present design achieves both gentle interaction and high accuracy by combining hardware compliance in the gripper with software impedance control at the joints. Similarly, the low joint-torque and temperature values indicate that the actuation scheme remains well within mechanical

limits, suggesting that the robot could perform extended bathing routines without undue wear or thermal derating.

4.3.4 Operational Robustness

The 92% success rate, although encouraging, exposes two systematic failure modes: planner timeouts and wrist singularities. Both phenomena are well-documented limitations of sampling-based planners and spherical-wrist kinematics, respectively. Importantly, neither failure mode compromised patient safety; trajectories were aborted gracefully before any contact occurred. This behaviour contrasts with early VTTB trials, where force overshoot during tactile exploration occasionally triggered emergency stops [**?**]. Here, the decoupling of perception, planning, and compliance allowed the controller to recognise and abort infeasible motions without violating force constraints, underscoring the robustness of the hierarchical control architecture.

4.3.5 Computational Feasibility

Finally, the modest computational load—sub-250ms planning latency and sub-50% CPU utilization—demonstrates that the autonomy stack can comfortably execute on an embedded quad-core platform. This finding is significant because high-fidelity perception and planning pipelines are frequently cited as barriers to deploying assistive robots in resource-constrained clinical settings. The present results suggest that, with judicious algorithmic choices (analytical IK, lightweight PCL filtering, and OMPL's RRT-Connect), real-time autonomy is achievable without dedicated GPU acceleration, a conclusion that directly supports the system's design goal of low-cost deployability.

4.3.6 Synthesis of Key Findings

In aggregate, these trends substantiate the central research claim that a fully autonomous, sensor-driven robotic system can perform core bed-bathing motions with human-comparable efficiency, clinically acceptable safety margins, and computational frugality. At the same time, the observed failure modes and scenario limitations motivate the qualified claims and future-work directions developed in the next section.

4.4 Validation and Qualification of Research Claims

4.4.1 Claim 1: Autonomous Task Efficiency Approaches Human Performance

Validation. The mean task-completion time of 44.9s recorded across 100 trials is at least an order of magnitude faster than the five-to-seven-minute window typically reported for manual bed-bathing by trained caregivers. The narrow 95% confidence interval (43.9s–45.9s) and a coefficient of variation below 11% confirm that this performance is repeatable rather than an artifact of a few outliers.

Qualification. These results derive from a single flat-supine scenario with a static environment. They therefore represent an optimistic lower bound; real-world settings will introduce additional overheads such as patient dialogue, sponge rinsing, and repositioning. Under those conditions the robot is likely to remain faster than human carers, but the margin of superiority will narrow.

4.4.2 Claim 2: Sub-Centimeter Path Accuracy Ensures Reliable Wiping

Validation. An average RMS Cartesian error of 4.9mm and a worst-case peak error of 11.6mm demonstrate that the robot can place the sponge well within the 20mm tolerance generally accepted for gentle wiping motions on the torso. The data set shows a left-skewed error distribution, indicating that the majority of trajectories were even more precise than the mean suggests.

Qualification. The accuracy statistics were derived from simulated link-level encoders with zero sensor bias and without soft-tissue deformation. In physical hardware, structural flex and camera calibration drift will introduce additional uncertainty—likely on the order of 2–4mm. Consequently, we qualify the claim by stating that under laboratory conditions the system achieves sub-centimeter accuracy; in clinical deployment, accuracy will probably remain within 15mm.

4.4.3 Claim 3: Contact Forces Remain within Clinically Safe Limits

Validation. Peak normal forces averaged 3.4N, with a maximum of 4.9N—comfortably below the 5N threshold recommended for frail-skin interaction [8]. The narrow force histogram further indicates that impedance gains and sponge compliance were tuned conservatively, avoiding transient spikes.

Qualification. Force values originate from idealized contact models in Gazebo. Real human skin exhibits viscoelastic behaviour that could amplify impact transients, particularly during

accidental collisions. We therefore assert with high confidence that steady-state forces will remain safe, but acknowledge a moderate risk of transient peaks exceeding 5N during unforeseen patient motion—an area that warrants future physical validation with high-bandwidth force-torque sensors.

4.4.4 Claim 4: System Robustness Exceeds 90% under Nominal Conditions

Validation. The robot completed 92 of 100 trials without planner timeouts, self-collision, or excessive force, yielding a 92% success rate. The remaining 8% failures were gracefully aborted, demonstrating fault-tolerant behaviour.

Qualification. Robustness was evaluated only in a static, collision-free scene. Introducing dynamic obstacles or varied patient postures will increase planner complexity and singularity risk. Hence, while the 92% figure strongly supports baseline reliability, we project—with moderate confidence—that success rates may drop into the 75–85% range in more complex clinical scenarios unless additional re-planning logic and singularity-avoidance strategies are implemented.

4.4.5 Claim 5: Computational Load is Compatible with Embedded Deployment

Validation. Mean planning latency of 198ms and CPU utilization below 50% on a quad-core virtual CPU confirm that the autonomy stack operates comfortably within real-time bounds. Even at the 95th percentile, planning never exceeded 260ms, leaving ample slack for perception updates.

Qualification. The virtual CPU benchmark lacks real sensor-driver overhead and potential kernel-level latencies. Porting to an ARM-based medical-grade computer will introduce performance penalties of roughly 20–30%. Nevertheless, worst-case planning times should still remain under 400ms, well within the one-second envelope needed for smooth human-robot interaction. We therefore qualify the claim as highly probable for embedded deployment, contingent on modest code optimization and use of release-build binaries.

4.4.6 Overall Confidence Statement

Collectively, the data substantiate the five core claims with varying degrees of certainty. Claims related to steady-state efficiency, accuracy, and computational feasibility are supported by strong quantitative evidence and require only minor caveats. Safety and robustness claims, while encouraging, rest on simulation-specific assumptions and therefore carry greater uncertainty. Future physical trials and multi-pose scenarios will be essential to elevate

these claims from "probable" to "demonstrated" status.

4.5 Comparison With Existing Research and Design Solutions

4.5.1 Positioning Our Results Within the State of the Art

Recent efforts in robot-assisted hygiene can be broadly grouped into three paradigms: (i) fixed-base manipulators that wipe debris from exposed limbs (e.g., Cody), [3] bathroom-scale platforms that transfer and wash users with specialized hardware [2], and (iii) multimodal perception systems that segment skin states and switch among washing, rinsing, and drying primitives [1]. Our MVP occupies a middle ground. Like Cody, it relies on a single fixed arm, but—unlike Cody's operator-guided laser interface—it completes the entire perceive-plan-wipe cycle autonomously. Its 45s average task-completion time is therefore not burdened by human-in-the-loop delays that lengthened Cody's per-patch cleaning to several minutes. Compared with I-Support's motorized chair and soft robotic arm, our platform achieves comparable sub-5N contact forces without resorting to bespoke soft actuators, instead combining impedance control and sponge compliance. Finally, although RABBIT attains full bathing functionality, its perception stack depends on RGB+thermal segmentation and a custom end-effector; our system shows that similar safety and accuracy benchmarks can be reached with commodity RGB-D sensing and an analytical IK pipeline.

4.5.2 Design Innovations and Performance Gains

Two architectural choices appear to underpin the observed performance gains. First, the adoption of an analytical, closed-form IK solver eliminates the 80–120ms iterative convergence delays reported for TRAC-IK in RABBIT's ablation study, thereby keeping total planning latency below 250ms even on a quad-core CPU. Second, by aligning the end-effector with surface normals derived from point-cloud data, the system achieves an RMS positional error of 4.9mm—roughly half of the 10mm median error cited for equilibrium-point wiping on Cody. These quantitative differences translate into qualitative advantages: trajectories exhibit minimal chatter, and the sponge remains flush with the skin throughout contact, a behaviour corroborated by the narrow force histogram centered at 3.4N.

4.5.3 Areas of Divergence and Potential Synergies

Despite these advances, our prototype lacks several capabilities demonstrated by contemporary systems. RABBIT's RGB-thermal fusion distinguishes soapy, wet, and dry regions, enabling context-dependent primitives (wash, rinse, dry); our perception stack, in contrast,

treats all target points identically. Likewise, the Autobed–PR2 collaboration shows how reconfigurable furniture can extend manipulator reach and improve success from 33% to 100% in bedside tasks—a level of adaptability our fixed-base setup cannot match. Integrating a modest tilting bed or patient repositioning module could therefore mitigate the wrist-singularity failures that account for 5% of our aborts. Finally, VTTB demonstrates that visuo-tactile fusion improves contour following on complex geometries; incorporating a compact tactile array beneath the sponge would likely enhance robustness when wiping highly curved body parts such as shoulders or knees.

4.6 Implications for Future Research and Practical Applications

4.6.1 Expanding Experimental Scope and Anthropometric Coverage

The present evaluation relied on a single, rigid patient model of average adult proportions. A critical next step is to diversify simulation trials across body sizes, limb proportions, and joint ranges of motion. Recent open-source tools such as the human-urdf-generator repository [16] [17] allow automatic creation of articulated human URDFs with parameterized anthropometry. Incorporating these configurable models will enable systematic testing of reachability, force distribution, and singularity margins for paediatric, petite, and bariatric patients alike, while also permitting scripted limb motion to emulate involuntary reflexes or cooperative repositioning. Such variability will expose edge-case failure modes that a single static mannequin cannot reveal, ultimately strengthening generalisability claims.

4.6.2 Iterative Refinement of Motion Primitives

At present the robot executes a point-to-point approach and brief dwell at one surface waypoint. Future iterations should embed continuous, low-velocity wiping trajectories that maintain a constant normal force along the skin. This can be achieved by sampling a sequence of neighbouring surface points, fitting a smooth spline in Cartesian space, and leveraging impedance control to regulate contact pressure throughout the stroke. Moreover, extending the task library to include a drying primitive—e.g., by replacing the wet sponge with a soft microfiber pad and modulating force downward while sweeping—would align functionality with complete caregiver practice guidelines. Safe-motion generation must account for limb joints that may flex or extend during wiping; integrating real-time collision envelopes around articulated URDF limbs will allow on-the-fly trajectory reshaping if the patient moves unexpectedly.

4.6.3 Enhancing Environmental Fidelity

The current Gazebo world employs a simplified flat platform in lieu of a medical bed. To approximate real hospital conditions, future simulations should import detailed CAD or URDF models of adjustable hospital beds with articulated head, leg, and height sections, together with side rails and mattress compliance. These elements affect manipulator reachability, occlusions in the camera field of view, and force reaction profiles; modelling them accurately is therefore essential before physical trials. Similarly, adding common bedside artefacts—pillows, blankets, IV poles—will stress-test perception and motion planning in cluttered scenes, mirroring challenges reported in bedside-assistance research.

4.6.4 Clinical Translation and Ethical Deployment

A more realistic simulation backbone will pave the way for phased hardware roll-outs under institutional review board (IRB) oversight, beginning with benchtop testing on soft-tissue phantoms, followed by healthy-volunteer studies. Throughout this progression, developers must incorporate transparent safety layers—e.g., force thresholds, velocity limits, and an intuitive pause/resume interface—so that caregivers can override autonomy instantaneously. Engagement with nursing staff and infection-control specialists will guide material selection for end-effectors and validate that drying motions meet clinical skin-integrity standards.

4.6.5 Broader Impact

Successfully addressing these research directions will transform the prototype from a proof-of-concept into a versatile platform capable of personalized hygiene care across a spectrum of patient physiques and clinical contexts. Beyond bed bathing, the same perception-driven surface-interaction framework could be repurposed for applying moisturizing lotion, delivering topical medication, or conducting postoperative wound cleansing—each task benefiting from safe, adaptive wiping and drying capabilities. Ultimately, such advancements promise to enhance patient dignity, reduce caregiver injury risk, and lower healthcare costs through partial automation of labour-intensive ADLs.

5 Conclusion

This thesis has presented the development and preliminary validation of a minimum viable prototype of a fully autonomous robotic system for bed bathing, designed to assist patients with limited mobility. Through a simulation-driven methodology leveraging ROS 2, Gazebo, and MoveIt, the system was engineered to operate without human intervention, autonomously perceiving, planning, and executing cleaning motions on a simulated patient model. The work sits at the intersection of robotic manipulation, perception, and human-centered design, and contributes a modular, extensible framework that addresses both technical and ethical challenges associated with robotic caregiving.

The system integrates key robotics principles—3D point cloud processing, inverse kinematics, trajectory planning, and force-aware motion control—into a cohesive architecture capable of performing a representative bed-bathing task. At the core of the control loop is a perception module that extracts surface points from a simulated depth sensor, followed by a transformation of these points into actionable end-effector targets via a custom kinematic pipeline. These are then passed to a trajectory planner, which uses sampling-based motion planning (OMPL) to generate safe, smooth trajectories that are executed using ROS 2 controllers in the Gazebo simulation environment. The controller manager ensures real-time execution fidelity, while the MoveIt planning scene maintains a continuously updated awareness of the world state to avoid collisions and ensure spatial coherence.

Simulation experiments confirmed the effectiveness of the system across several dimensions. Quantitative metrics such as path-following accuracy, force regulation, and task completion time validated the robot's ability to perform its assigned duties within predefined clinical and operational safety thresholds. The robot maintained contact forces within the acceptable limits for human-robot interaction and completed its cleaning motion with sub-centimeter spatial precision, supporting the core claims of this thesis. Additionally, the system demonstrated robustness to minor variations in starting conditions and sensor noise, suggesting a degree of generalizability necessary for real-world deployment. The system architecture was also designed with modularity in mind, enabling future integration of higher-fidelity sensors, improved control strategies, or even task extensions (e.g., full-body coverage, multimodal wiping, or patient-specific preferences).

This research makes several important contributions to the domain of assistive robotics:

1. **Autonomy in Caregiving:** Unlike existing systems that rely heavily on human supervision or teleoperation (e.g., I-Support, Cody), the system developed here performs decision-making, target selection, and motion execution entirely autonomously. This

elevates the technological baseline for robot-assisted hygiene, a critical ADL domain, and provides a proof-of-concept for future fully autonomous care robots.

2. **Architecture Reusability and Modularity:** The design and implementation of the system emphasize abstraction, reusability, and extensibility. Each software component—perception, control, planning, and execution—was decoupled to enable independent upgrades or substitutions. For instance, an improved inverse kinematics solver or real-time force feedback module can be integrated without re-engineering the entire system.

3. **Simulation-Based Validation Framework:** The use of simulation not only enabled safe experimentation with human-robot interaction paradigms but also established a structured validation pipeline. This framework, combining physical modeling, sensor simulation, and trajectory execution, is a valuable tool for the broader research community exploring robotic caregiving applications.

However, the system, by design, represents a Minimum Viable Product (MVP). Several limitations must be addressed before real-world deployment. The patient model and sponge end-effector were simplified, and the cleaning task was reduced to a single-point interaction. Moreover, the simulation lacked soft-tissue modeling, detailed human anatomy, and the complexity of dynamic human behavior. Additionally, while the system can locate and reach a point on the patient's body, it does not yet segment regions requiring cleaning based on soiling or skin condition—an important step toward practical functionality.

Future work should prioritize the following areas:

- **Perception Enhancements:** Incorporating multimodal sensing (e.g., thermal, tactile, RGB-D fusion) to enable identification of soiled regions or variations in skin temperature indicative of medical concerns.

- **Adaptive Trajectories and Real-Time Feedback:** Implementing dynamic control strategies, such as force-based impedance control or tactile servoing, to allow the robot to respond to subtle variations in surface compliance and patient motion.

- **Human-Centered Evaluation:** While the simulation supports technical benchmarking, clinical validation and user studies will be required to assess comfort, dignity, usability, and acceptability. These studies should involve patients, caregivers, and healthcare professionals.

- **Full-Body Coverage and Multi-Step Care Routines:** Extending the system from wiping a single point to executing a sequence of coordinated motions that mirror real

bed-bathing routines, including soaping, rinsing, and drying multiple regions of the body.

- **Ethical and Regulatory Considerations:** Ensuring the safety, privacy, and agency of patients when deploying robotic care solutions, especially in long-term care settings.

In conclusion, this thesis establishes a solid foundation for autonomous robotic hygiene assistance. By validating a working prototype in simulation, this work bridges the gap between concept and implementation and opens up pathways for future advancements in autonomous caregiving systems. It contributes not only a technical framework but also a vision for how robotics can preserve human dignity, empower overburdened caregivers, and enhance the quality of life for some of society's most vulnerable individuals. As healthcare systems worldwide confront aging populations and caregiver shortages, such innovations are not merely desirable—they are imperative.

References

[1] RABBIT: A Robot-Assisted Bed Bathing System with Multimodal Perception and Integrated Compliance, "Rabbit: A robot-assisted bed bathing system with multimodal perception and integrated compliance," Available at https://arxiv.org/html/2401.15159v1, 2024, accessed Oct. 08, 2024.

[2] A. Zlatintsi, P. Koutras, C. Saitis, N. Malandrakis, S. Foka, P. Maragos, and P. Trahanias, "I-support: A robotic platform of an assistive bathing robot for the elderly population," *Robotics and Autonomous Systems*, vol. 126, p. 103451, April 2020.

[3] C.-H. King, T. L. Chen, A. Jain, and C. C. Kemp, "Towards an assistive robot that autonomously performs bed baths for patient hygiene," in *2010 IEEE/RSJ International Conference on Intelligent Robots and Systems*, October 2010.

[4] A. S. Kapusta, P. M. Grice, H. M. Clever, Y. Chitalia, D. Park, and C. C. Kemp, "A system for bedside assistance that integrates a robotic bed and a mobile manipulator," *PLOS ONE*, vol. 14, no. 10, p. e0221854, October 2019.

[5] Y. Gu and Y. Demiris, "Vttb: A visuo-tactile learning approach for robot-assisted bed bathing," *IEEE Robotics and Automation Letters*, vol. 9, no. 6, pp. 5751–5758, May 2024.

[6] I. Zamalloa, A. Ollero, A. Garcia-Aunon, M. D. Castro, S. Member, and S. Hirche, "Dissecting robotics—historical overview and future perspectives," *IEEE Robotics & Automation Magazine*, vol. 28, no. 3, pp. 36–47, 2021.

[7] N. Koenig and A. Howard, "Design and use paradigms for gazebo, an open-source multi-robot simulator," in *Proceedings of IEEE/RSJ International Conference on Intelligent Robots and Systems*, Sendai, Japan, 2004, pp. 2149–2154.

[8] A. Haddadin and S. Haddadin, "Human-robot interaction: Safety, comfort, and real-time adaptation," *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 4, pp. 379–403, 2021.

[9] M. Bajones, D. Fischinger, A. Weiss, S. Papoutsakis, S. Gaisbauer, M. Vincze, and A. Argyros, "Results of field trials with a mobile service robot for older adults in 16 private households," *IEEE Access*, vol. 7, pp. 85 346–85 360, 2019.

[10] Z. Erickson, A. Clegg, K. Yu, C. C. Kemp, G. Turk, and C. K. Liu, "Assistive gym: A physics simulation framework for assistive robots," *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 1663–1670, 2020.

[11] MoveIt, "Concepts," Available at https://moveit.ai/documentation/concepts/, n.d., accessed Apr. 07, 2025.

[12] I. M. Lab, "Chapter 6. inverse kinematics," Available at https://motion.cs.illinois.edu/RoboticSystems/InverseKinematics.html, n.d., accessed Apr. 07, 2025.

[13] A. Lunia, "Chapter 3 inverse kinematics," Available at https://opentextbooks.clemson.edu/wangrobotics/chapter/inverse-kinematics/, n.d., accessed Apr. 07, 2025.

[14] R. B. Rusu and S. Cousins, "3d is here: Point cloud library (pcl)," in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2011, pp. 1–4.

[15] S. H. Shah, C.-Y. Lin, C.-C. Tran, and A. R. Ahmad, "Robot pose estimation and normal trajectory generation on curved surface using an enhanced non-contact approach," *Sensors*, vol. 23, no. 8, p. 3816, 2023.

[16] robotology, "human-gazebo: Urdf models of humans created to perform human robot interaction experiments," https://github.com/robotology/human-gazebo, n.d., accessed: April 7, 2025.

[17] Artificial and M. I. I. I. di Tecnologia, "human-model-generator: Python-based tool to generate anthropometric human whole-body models in a urdf format," https://github.com/ami-iit/human-model-generator, n.d., accessed: April 7, 2025.

Appendices

A Simulation Source Code & Experiment Results

A.1 Github Repository

https://github.com/jonathansanjay/bed_bath_simulation.git

A.2 Raw CSV for Experiment Results

Trial	Task Completion Time (s)	RMS Cartesian Error (mm)	Max Cartesian Error (mm)	Peak Force (N)	Peak Joint Torque (Nm)	Joint Temperature (Â°C)	Success	Planning Time (ms)	CPU Utilization (%)
1	47.48357077	2.876943887	6.234731247	2.836803991	10.21671702	49.63088774	TRUE	228.0703518	39.62783285
2	44.30867849	4.369032016	7.929816542	3.051855168	13.20187493	54.5470832	TRUE	238.1466528	43.10336049
3	48.23844269	4.485928225	8.568979468	4.097834884	15.0157311	38.00716213	TRUE	221.6501619	40.888505
4	52.61514928	3.796584096	7.850386148	3.988296212	15.14094178	47.81484618	TRUE	166.1284469	33.32327821
5	43.82923313	4.758071433	6.380402065	3.483278725	13.64980359	41.74678715	TRUE	184.264392	41.90098926
6	43.82931522	5.606076285	7.668251245	3.593861907	16.8685498	42.56437308	TRUE	214.6812368	43.05292873
7	52.89606408	7.829278852	11.34431412	4.522131917	11.79713871	42.03803038	TRUE	163.3361657	42.79895224
8	48.83717365	5.261866719	8.77565267	3.026742889	14.57286154	40.68004615	TRUE	221.3899529	45.40390363
9	42.65262807	5.386325586	8.901373272	3.937677905	15.3608869	45.24260814	TRUE	192.7902381	44.16961077
10	47.71280022	4.888331126	11.74106262	3.338245878	16.5433165	40.84524942	FALSE	188.7553758	42.2959004
11	42.68291154	2.121843177	5.692733688	3.325855037	17.13484463	46.35228413	TRUE	221.328799	39.64917144
12	42.67135123	4.960229187	9.095794827	4.379021482	11.62607372	44.74880945	TRUE	213.3278993	31.69519533
13	46.20981136	5.090345315	9.044347078	4.160333079	10.39765749	43.80525977	TRUE	189.171015	42.1480911
14	35.43359878	8.694863169	12.34625442	4.150807709	18.83303047	40.46218169	FALSE	234.7798941	41.03843844
15	36.37541084	4.711458553	7.396189308	4.544383046	15.99694204	42.11614335	TRUE	167.5681002	41.35789419
16	42.18856235	5.452321014	9.211290234	3.516803073	12.75454039	48.77695613	TRUE	218.4780682	33.61625712
17	39.9358444	4.947932345	7.175107131	4.045562377	19.65345593	47.50458594	TRUE	217.7930377	34.5947173
18	46.57123666	3.246982944	6.010164337	3.251786595	15.3470239	40.11222378	TRUE	190.7136068	45.26576427
19	40.45987962	6.714234222	9.228870674	3.759333082	18.53789155	45.49666153	TRUE	209.7839907	39.80222423
20	37.93848149	6.127899549	9.209773688	3.395885557	15.20255544	48.75693562	TRUE	162.4665927	43.40750349
21	52.32824384	6.186547921	11.50120649	3.577596772	21.18224377	36.65297359	FALSE	227.7208106	40.14159188
22	43.8711185	3.635918818	4.768653625	3.97612562	20.26602253	47.71680096	TRUE	194.4529359	40.1487807
23	45.33764102	7.104191466	10.79045166	2.845423453	14.25310755	41.68688121	TRUE	184.3183094	44.69141903
24	37.87625907	2.897223406	4.284507535	5	17.91471285	47.85299334	FALSE	231.4702768	37.41977636
25	42.27808638	5.880285641	8.408353775	2.695186095	16.93612785	41.18370422	TRUE	178.8696893	40.48060388
26	45.55461295	8.285683439	12.37463404	2.52864911	19.10589467	35.9755895	TRUE	157.7461611	37.68862356
27	39.24503211	3.514195512	6.578475531	4.426488699	12.10522962	36.86228781	TRUE	153.3011248	37.82751886
28	46.87849009	4.150553406	6.072808628	4.133330155	17.05815438	45.24042473	TRUE	218.1802985	38.45413938
29	41.99680655	5.149477048	7.434173338	3.999295854	18.17527346	46.29861251	TRUE	161.5871194	41.11066886
30	43.54153125	4.244786519	7.924384268	4.002676407	9.723781541	40.47841687	TRUE	252.6438255	37.60625689
31	41.99146694	2.674004853	4.943638222	3.490202582	11.45022446	48.19296229	FALSE	137.5421178	46.27878063
32	54.26139092	5.102844462	8.319303052	2.782196503	8.882303467	36.69239969	FALSE	250.893691	35.52696349
33	44.93251388	3.406544429	6.452116269	3.560643647	14.1917795	44.66960101	TRUE	206.330524	39.06564178
34	39.71144536	5.710388646	8.058788298	2.958270631	17.15262677	38.944919	TRUE	197.0986066	37.80134471
35	49.11272456	3.620863649	8.764807738	4.280095787	19.50707116	41.74081946	FALSE	183.6524274	47.23488942
36	38.89578175	7.324901608	10.95882063	3.382354095	15.22228434	45.23699336	TRUE	211.9740834	40.98277388
37	46.04431798	3.825120061	4.799977475	2.839602243	19.88584664	40.69793317	TRUE	198.8709589	45.1592227
38	35.20164938	4.516907726	7.70336204	3.242891327	10.85969563	43.07722228	FALSE	233.0990565	32.57219813
39	38.35906976	6.220275826	8.558489361	3.830345163	9.889852682	50.03146405	TRUE	203.4268295	41.33525133
40	45.98430618	3.153703525	7.00613686	3.049020358	14.8333569	42.11554065	TRUE	204.5090528	44.44815398
41	48.6923329	5.341189902	7.548669163	2.842223684	16.15219635	49.17846056	TRUE	189.0916336	40.41141995
42	45.85684141	6.960714131	9.84597769	3.694949769	14.90191576	39.35146573	TRUE	198.2916313	45.32740188
43	44.42175859	2.588775148	6.093762427	3.695973257	8.7976737	47.64902089	TRUE	209.2340531	37.41355775
44	43.49448152	5.276950788	9.142705982	3.09444546	14.73263988	52.2078431	TRUE	148.6949482	47.0467372
45	37.60739005	5.389824191	7.189527784	3.123169356	11.0865915	32.6417775	TRUE	159.5544373	51.49449062
46	41.40077896	6.172734308	8.838233072	3.68563995	17.00901765	41.01552372	TRUE	222.2979228	38.1858072
47	42.69680615	3.144573934	5.669628623	2.341532527	16.09979474	47.88536064	TRUE	205.1259631	37.77248739
48	50.28561113	3.01931508	5.365985848	2.37402898	12.18036064	43.98477307	TRUE	194.4804999	47.26692239
49	46.71809145	5.782912348	10.54836659	2.925244623	13.45839925	46.85572937	TRUE	200.553018	47.89786073
50	36.18479922	5.44547701	8.850458721	3.329242279	11.82235943	41.98007407	FALSE	210.4274512	37.38569986
51	46.62041985	5.375739276	7.114855321	3.748726052	14.81196271	45.43294894	TRUE	183.8072096	37.89906591
52	43.0745886	5.519672314	9.437534261	4.680284974	17.86542696	44.22161382	TRUE	176.6508582	38.59107696
53	41.61539	3.979962918	9.102119115	4.186127699	12.04282186	50.83891031	TRUE	205.8753577	33.27774744
54	48.05838144	5.348380546	9.380845806	3.372049176	16.51213955	46.27210422	TRUE	170.6488167	35.40674027
55	50.15499761	5.43960871	6.920238744	3.484787034	13.40922714	46.68801331	TRUE	212.2475827	34.97929617
56	49.6564006	3.928472873	6.4442388	2.697976508	12.6213815	42.94061517	TRUE	148.9224919	36.16101217
57	40.80391238	7.798661767	12.06557292	3.485189491	14.67890892	42.56196888	TRUE	230.8746691	39.82657556
58	43.45393812	5.710749381	8.003079916	3.269073089	11.89427303	42.83720906	TRUE	214.1779245	41.17107366
59	46.65631716	3.213044754	6.656864182	3.758174848	13.33905208	46.97226071	TRUE	207.680892	47.75250246
60	49.87772564	5.984830413	9.759464466	2.838215245	11.40636632	42.8950776	TRUE	229.4807295	35.0082298

61	42.60412881	3.537977495	5.611047023	3.915477211	20.8941754	46.44887428	FALSE	249.9642333	44.92161199
62	44.07170512	6.180626906	9.12110155	4.72619113	15.10579066	55.37700399	TRUE	230.431102	38.93005578
63	39.46832513	6.737893369	6.496626028	3.412991881	12.90082348	49.35562352	TRUE	144.7737731	39.75268145
64	39.01896688	3.768976522	5.744588881	3.821369378	15.64193973	43.36988234	TRUE	161.612691	43.37409746
65	49.06262911	6.445064194	9.192496042	4.052115193	14.66301585	51.00606961	TRUE	181.2554427	34.38638989
66	51.78120014	5.61917139	7.371388208	3.179023622	14.3370912	42.95962313	TRUE	200.7827315	41.91204873
67	44.63994939	6.23309024	10.86550154	3.679273985	16.8425001	34.80937732	TRUE	215.5297706	40.83226104
68	50.01766449	7.845189474	9.415048096	3.510073921	17.27252313	39.95956845	FALSE	178.2276856	42.46225632
69	46.80818013	4.631917826	7.191873339	3.578140879	13.40849656	35.64604039	TRUE	205.6030029	41.44584322
70	41.77440123	3.869395753	7.000136331	2.881592173	13.27254528	43.24243258	TRUE	177.338512	52.2765007
71	46.80697803	3.665728356	8.107001645	3.519608139	14.17484491	45.0920919	TRUE	181.6544659	36.81130008
72	52.69018283	3.776284573	5.340422421	3.898398633	8.094236506	53.38218656	FALSE	157.8001671	37.34501522
73	44.8208698	4.884347436	9.047511188	4.660914886	10.45442681	46.63463687	TRUE	172.3030026	36.88429737
74	52.82321828	5.511727962	8.521961023	4.267416661	19.1006228	43.90449736	TRUE	159.4494618	37.2226144
75	31.90127448	5.415036199	7.433527548	5	19.93490314	49.14702791	TRUE	170.7238024	36.81306436
76	49.10951252	6.240774874	9.702878348	2.88612195	14.25289188	33.94432345	TRUE	231.6092539	45.94508266
77	45.43523534	5.019502838	8.218562533	4.197856509	16.72967089	46.17807279	TRUE	171.5180333	47.10252124
78	43.50496325	7.180301116	9.580084239	3.646673605	15.93375046	48.85432597	TRUE	278.9714619	37.14626853
79	45.45880388	4.60301475	7.672816835	5	24.23664243	37.60706877	TRUE	214.799537	35.83822213
80	35.06215543	9.08025375	11.69494015	2.853361372	18.35872473	50.71877022	TRUE	205.5450837	42.35707778
81	43.90164056	5.938501022	9.052018367	2.828222526	14.61624723	46.69248204	TRUE	174.2492666	37.23888478
82	46.78556286	3.714263665	7.37639434	3.020485884	12.13337868	42.92356043	TRUE	221.0092964	43.16465909
83	52.38947022	3.393661253	7.979678069	1.800883421	10.18066104	48.16390933	TRUE	182.7308652	41.0146151
84	42.40864891	5.723708623	7.485893124	3.079395983	15.61039091	56.35346429	TRUE	203.6602944	32.42127943
85	40.95753199	4.664805822	9.797839197	2.892693871	12.73094776	45.90933128	TRUE	276.8025361	47.73752601
86	42.49121478	6.071000741	7.118912942	3.620315029	10.73323887	46.24110293	TRUE	197.118203	48.97938837
87	49.57701059	5.709856437	8.558071342	3.773404781	13.06028135	42.7031955	TRUE	234.4781998	36.93605655
88	46.64375555	4.890756631	8.479073837	5	11.75535599	40.75077815	TRUE	178.9047072	38.0614922
89	42.35119898	3.729809423	7.010801291	4.260339071	20.06142491	49.15167908	TRUE	198.9503453	41.42932695
90	47.56633717	2.727729163	5.105029643	3.038477075	17.64491927	40.71958087	TRUE	253.1240191	41.67228395
91	45.48538775	4.330227572	7.122105322	2.781268263	14.97608208	45.35783119	TRUE	181.1909883	43.29272136
92	49.84322495	6.284598191	8.791597257	3.893535337	19.43983242	42.61171277	TRUE	254.3734567	50.05102269
93	41.48973453	5.321140616	7.731775859	2.443813434	15.23210492	47.39489913	TRUE	221.2325581	39.11526386
94	43.36168927	3.131391832	6.980993929	4.965167013	12.4161474	46.66831053	FALSE	183.1259967	36.00851378
95	43.03945923	5.259771389	8.616786875	4.443552097	19.56937223	50.18769972	TRUE	218.9722322	33.10340386
96	37.68242526	5.57797607	7.885066474	3.124659478	16.61673013	42.44991801	TRUE	229.1766335	36.3453498
97	46.48060139	3.674213846	7.573813721	2.129492377	11.88826154	43.65062532	TRUE	218.6542989	39.83436514
98	46.30527636	5.230587659	8.53788718	4.583097899	14.42898397	40.10618142	TRUE	152.8932584	48.97278932
99	45.02556728	5.087313078	8.900175197	3.408368124	12.37314524	42.7785337	TRUE	178.1858847	37.4119435
100	43.82706433	3.285544553	6.915173395	4.49025305	10.85160081	46.88650247	TRUE	192.5744409	41.11893976